Certified bounds and exact search for transit route network design

Tianqin Jiao

Independent researcher

Abstract

Heuristics for the Mandl and Mumford transit route network design benchmarks have produced steadily better networks, but for the standard problem on the four Mumford instances only the street shortest-path bound was known. We compute certified lower bounds for this problem. We relax the problem by replacing passenger flows with short path lists and price routes by dynamic programming over a string-matching automaton. A computable pricing bound keeps the Lagrangian bound valid even if pricing stops early. Complete route pools certify a bound for every dual point in a box, and cutoff pools limit branching to routes that could improve a network. A branch-and-price search over routes, path groups and route classes evaluates integer networks exactly. Completed trees are sound, and under stated conditions the search terminates finitely. Our search proves the twelve known Mandl optima, and on Mumford0–3, our networks beat the best printed values by at least 0.67–1.36%. The certified gaps are at most 0.28, 1.29, 2.04 and 2.79%, which closes 62–95% of the street-bound gap. An exact search that would close Mumford0 is also running.

Keywords: Transportation, Transit network design, Column generation, Branch-and-price, Lower bounds

1. Introduction

The transit route network design problem, or urban transit routing problem, chooses S routes on a street network. The benchmark model minimizes the demand-weighted average travel time with a penalty for each transfer, under fixed street travel times and without frequencies.

On the networks of Mandl (1979, 1980) and Mumford (2013), evolutionary algorithms, hyper-heuristics and neighbourhood search have found steadily better networks. However, how far any of them is from optimal has been unclear. For the standard problem on the Mumford instances, the only published lower bound we know of is the street shortest-path bound (Mumford, 2013, Table II). It ignores that only S routes exist and that transfers cost time, so it lies far below every feasible network. Against such a bound, one cannot tell whether a small improvement closes most of the remaining gap or barely any of it.

Exact methods solve Mandl's network, where all legal routes can be listed (Gao et al., 2026; Vermeir et al., 2021). Starting from the size of Mumford0, Vermeir et al. (2021, §4.7) report that precomputing all lines already exceeds memory and suggest generating lines during the search. We generate routes by column generation, and pricing is then the obstacle. A route earns value from whole passenger journeys that it carries, directly or as one leg of a transfer, so pricing is not a shortest-path problem.

Our key step is therefore to change what the master problem sees (Section 4). We replace passenger flows by a short list, or menu, of direct and one-transfer paths for each pair, with paths

Email address: jtq@umich.edu (Tianqin Jiao)

grouped by length, and charge every other journey an optimistic fallback cost. A route then earns value only from the nodes it visits and the listed paths it contains, which a pricer can score while it extends the route. The fallback keeps the relaxation valid however short the menu (Proposition 1). Short menus do cost bound strength, and will be measured properly.

Three further ideas make this relaxation computable, certifiable and exact.

Automaton pricing (Section 5). Containing a listed path is a pattern-matching question, so a dynamic program over an Aho–Corasick automaton recognizes listed paths while it extends a route. Its walks may repeat nodes or collect a path prize twice, and we penalize repeats with nonnegative multipliers. Any such multipliers give a computable bound on the best route prize, and hence a valid master bound when pricing stops early (Proposition 2). Each evaluation of this bound is one dynamic program with at most $\ell_{\max}$ passes over the automaton's transitions, so its cost grows with the menu rather than with the number of legal routes (3.4×10^9 on Mumford0 and an estimated 1.1×10^{17}–1.0×10^{19} on Mumford1–3). This is what makes networks beyond Mandl's size tractable. It also makes possible an exact search that can close Mumford0, still running (Section 8.6).

Complete pools (Section 6). Column generation tails off near a good dual point and may not reach the LP optimum. Inside a dual box, the routes that could price out form a threshold set that the same automaton enumerates completely. This gives a bound at every dual point of the box (Proposition 3), and the LP optimum itself when the box is inactive. With an improvement cutoff in place of the box, the same threshold gives a cutoff pool that contains every route of an improving network (Proposition 4), so the finisher of Section 7 branches only over those routes.

Exact search (Section 7). The relaxation underestimates true cost, so the search cannot trust it. We branch on routes, path groups and route classes, which pricing can respect. Every integer network is evaluated exactly. Then either the menus are refined where they underestimate it, or the network is excluded by branching. A finite-pool finisher closes nodes with small gaps. In the closure run it also adds $\{0, \frac{1}{2}\}$-cuts, which rest on an integral-menu lemma (Lemma 2). Completed trees are sound, and the search converges finitely when every node is processed exactly (Theorem 1).

The experiments (Section 8) recover the twelve Mandl optima for $S = 3, \ldots, 14$, and the search proves them without listing the route pool. On Mumford0–3 they give certified bounds and, with a separate local search (Appendix D), new best-known networks. To our knowledge, these are the first certified lower bounds for Mumford0–3 above the street bound for the standard problem in which all S routes are design variables. They close 95, 85, 73 and 62% of the gap between the street bound and our networks. Gao et al. (2026, Table 10) bound a different problem, in which one or two routes are added to fixed published networks. We certify the optima of both of its Mumford0 cases, closing the 0.62% gap of the two-route case. On Mumford0, every strictly better network must replace at least six of the twelve routes (Corollary 1). At our best Mumford2 and Mumford3 networks, about two thirds of the gap (62.6 and 68.1%) is menu underestimation.

Section 2 reviews related work and Section 3 fixes the conventions.

2. Related work

Transit network design spans route configuration, frequency setting, timetabling and passenger assignment (Ceder and Wilson, 1986; Durán-Micco and Vansteenwegen, 2022; Guihaire and Hao, 2008; Owais, 2026). Capacity or frequency decisions change both the passenger costs and the feasible set. We study the fixed-route-count passenger-time benchmark as those published networks can be compared directly with our bounds. It is the uncapacitated case of Goerigk and Schmidt (2017, §2.2), in which passenger routing is a shortest-path computation in the chosen network.

Heuristics supply the upper side of that comparison. They include metaheuristic, evolutionary, multi-objective and bee colony methods (Fan and Mumford, 2010; John et al., 2014; Mumford,

2013; Nikolić and Teodorović, 2013), demand-based route generation (Kılıç and Gök, 2014), selection hyper-heuristics (Ahmed et al., 2019), and multi-objective simulated annealing and NSGA-II (Hüsselmann et al., 2024). The evolutionary variant of the learned heuristics of Holliday et al. (2025) improved the Mumford2 and Mumford3 passenger values. The genetic algorithm with sub-objectives of Wens and Vansteenwegen (2025) reports the best printed passenger values known to us. None of these methods can say how far a network is from optimal. We bound that distance and certify regions with no better network.

Exact methods do measure that distance, but so far only on Mandl or on Mumford problems whose base network is fixed. Fiss and Ritt (2012) solved Mandl with two and three routes by a mixed-integer program, without the usual eight-stop limit (their optimal routes have up to 13 stops). Vermeir et al. (2021) established Mandl optima with up to 14 lines with an exact line-planning model with integrated passenger routing. Gao et al. (2026) proposed arc- and route-based models, shortest-path inequalities and a branch-and-cut algorithm that separates route selection from transfer-inclusive passenger evaluation. On Mumford they add one or two routes to the fixed networks of Wens and Vansteenwegen (2025). For these $(S+1)$- and $(S+2)$-route problems they report gaps of 0–4.93% to their bounds (Gao et al., 2026, Table 10). Their base values coincide with our evaluation of the same route sets. We draw on their shortest-path inequalities and evaluator-based decomposition. For the Mumford problem with all S routes free, we handle the implicit route universe by pricing and complete-pool certificates.

Other models already generate lines by column generation. Borndörfer et al. (2007) generate lines in a line-planning model. Borndörfer and Karbstein (2012) count direct connections exactly, charge every other passenger path one uniform transfer penalty and price passenger paths by column generation; our menu terms are of the same kind, with short exact path lists and a lower-bounding fallback. Chu (2018) develops a parallel branch-and-price-and-cut algorithm for bus network design and timetabling. Martínez-Quezada et al. (2026) use a column-generation heuristic for a bilevel model of network design and frequency setting with capacities, and Duan et al. (2026) generate lines heuristically for an integrated design of fixed-route transit, frequencies and on-demand connections. Cancela et al. (2015) point to column generation as a next step for their transit network design formulations. Heinrich et al. (2022) study the complexity of line planning when every simple path may be a line, Schmidt and Schöbel (2015) analyse the complexity of integrating passenger routing into line planning and identify polynomial special cases, and Hansen et al. (2026) study exact line planning over a line pool with frequency-dependent passenger costs, operating costs and lost demand. Our question differs: how to price elementary routes whose value comes from the journeys they carry, and what route universe and cost model the resulting bound certifies.

Each phase of our algorithm uses established components. Every dual point gives the Lagrangian bound of column generation (Lübbecke and Desrosiers, 2005, §§2.1, 3.1). Pricing recognizes covered paths with the Aho–Corasick automaton (Aho and Corasick, 1975). Its penalized walks are a state-space relaxation in the sense of Christofides et al. (1981), and its memory strings are the decremental state-space relaxation and ng-route technique (Baldacci et al., 2011; Righini and Salani, 2008). To finish the LP, we stabilize column generation by dual price smoothing (Wentges, 1997) with the mis-pricing schedule of Pessoa et al. (2013, 2018). We then apply BOXSTEP (Marsten et al., 1975) through primal artificial columns priced at the box edges. du Merle et al. (1999) generalize this with bounded, penalized artificial columns. The search uses branch-and-price branching (Barnhart et al., 1998), canonical cuts (Balas and Jeroslow, 1972), local-branching neighbourhoods (Fischetti and Lodi, 2003), reduced-cost route enumeration (Baldacci et al., 2008) and $\{0, \frac{1}{2}\}$-Chvátal–Gomory cuts (Caprara and Fischetti, 1996). Strong relaxations and reduced-cost fixing are central in related exact methods (de Lima et al., 2023). DeLuxing (Yang, 2025) and dual-path fixing (Yamagishi et al., 2026) use several dual solutions for fixing. Section 7.3 reports our test of dual-path fixing.

3. Problem and evaluation conventions

We follow the constraints of Mumford (2013, §II) and the instance parameters of Mumford (2013, Table II). Let $G = (V, E)$ be an undirected street graph with positive integer edge travel times t_e. A legal route is a simple path with $\ell_{\min}$ to $\ell_{\max}$ stops, and $|r|$ counts the stops of route r. The set $\mathcal{R}$ of legal routes identifies a path with its reversal. A feasible network $R \subseteq \mathcal{R}$ has exactly S distinct routes, covers every node of V, including nodes without demand such as Mandl's node 15, and has a connected union.

The demand matrix D is symmetric. We use the unordered positive-demand pairs $\mathcal{Q} = \{(o, d) : o < d,\ D_{od} + D_{do} > 0\}$, weights $w_{od} = D_{od} + D_{do}$ and total demand $W = \sum_{(o,d)\in\mathcal{Q}} w_{od}$. A journey rides selected routes in either direction and transfers at shared stops. Its generalized time is the in-route travel time plus $p = 5$ minutes per transfer. With $c_{od}(R)$ the minimum generalized time for (o, d), the objective is

$$\min_{R \text{ feasible}} \mathrm{ATT}(R), \qquad \mathrm{ATT}(R) = \frac{\mathrm{TTT}(R)}{W}, \qquad \mathrm{TTT}(R) = \sum_{(o,d)\in\mathcal{Q}} w_{od} c_{od}(R). \tag{1}$$

We certify bounds in the total travel time TTT (passenger-minutes) and report the average travel time ATT in minutes.

The evaluator computes $c_{od}(R)$ in the direct-link network of Vermeir et al. (2021, §4.3), analysed by Aktaş et al. (2024). Every pair of stops on a selected route is joined by a link with its in-route travel time, and p is added between consecutive rides. For the transfer shares d_0, d_1, d_2, d_{un} (demand with zero, one, two or more transfers) we break ties among minimum-time journeys by the fewest transfers, which does not change ATT. Mumford (2013, eq. (1)) and later work charge every journey its generalized time and only report the share d_{un} of journeys with more than two transfers, so our objective is the published one. None of our reported networks has such journeys (Table 6).

All travel times and p are integers. Demand entries are multiples of five, so each $w_{od} = 2D_{od}$ is a multiple of 10 on all five instances and $\mathrm{TTT}(R) \in 10\mathbb{Z}$ for every network. Section 7.5 uses this for strict cutoffs.

The bound to beat is the street bound, the passenger lower bound LBpass of Mumford (2013, Table II):

$$L_0 = \frac{1}{W} \sum_{(o,d)\in\mathcal{Q}} w_{od} s_{od}, \tag{2}$$

with street shortest-path distances $s_{ab} = \mathrm{dist}_G(a, b)$. It is valid because every journey is a street walk and transfer penalties are nonnegative. It ignores the limits on route count and route length and the transfers they force. Our bounds rest on these limits.

4. Coverage relaxations with short path lists

A route earns its value from whole journeys, directly or with a transfer, not from its edges. So we give each pair a short list of candidate paths and an optimistic fallback cost for every other journey. We call a pair's list with its fallback a *menu*. The menus D, SL and PL below differ in which journeys they list, and each gives a coverage model. A route then enters the model only through its nodes and the listed paths it contains, which a pricer can score while extending the route (Section 5).

4.1. Path groups and a direct-journey menu

For each relevant pair (a, b), we enumerate simple street paths by nondecreasing length. Paths of equal length form a group. Let $\mathcal{P}_{ab,m}$ be a retained group with length $c_{ab,m}$, and h_{ab} a lower bound on the length of every omitted simple a–b path. If the first K complete groups are retained, h_{ab} may be the next distinct length. A route covers a group, $a_{r,ab,m} = 1$, if it contains one of its paths as a contiguous subpath in either direction.

The direct menu (D) offers the retained groups of each pair. Omitted direct journeys and journeys with a transfer pay the fallback

$$\rho^D_{od} = \min\{h_{od}, s_{od} + p\}. \tag{3}$$

Let z_r select route r and $\theta_{od,m}$ be the fraction assigned to an offered direct group. With M_{od} the offered groups and $C_0 = \sum_{(o,d)\in\mathcal{Q}} w_{od}\rho_{od}$, the coverage model is

$$\begin{aligned}
\min \quad & C_0 - \sum_{(o,d)\in\mathcal{Q}} \sum_{m\in M_{od}} w_{od}(\rho_{od} - c_{od,m})\theta_{od,m}, && && (4)\\
\text{s.t.} \quad & \sum_{r\in\mathcal{R}} z_r = S, && && (5)\\
& \sum_{r\ni v} z_r \geq 1 && v \in V, && (6)\\
& \sum_{m\in M_{od}} \theta_{od,m} \leq 1 && (o,d) \in \mathcal{Q}, && (7)\\
& \theta_{od,m} \leq \sum_{r\in\mathcal{R}} a_{r,od,m} z_r && (o,d) \in \mathcal{Q},\ m \in M_{od}, && (8)\\
& 0 \leq \theta_{od,m} \leq 1, \quad z_r \in \{0,1\}. &&
\end{aligned}$$

The unassigned fraction pays the fallback. The model keeps node coverage but relaxes connectivity and detailed passenger assignment. Its LP relaxation replaces $z_r \in \{0, 1\}$ by $0 \leq z_r \leq 1$.

We raise the direct fallback by the direct-edge strengthening of Gao et al. (2026) when the unique shortest o–d path is the edge $\{o, d\}$. If a selected route uses the edge, the shortest direct ride is available. Otherwise every journey avoids the edge and is at least as long as the next simple-path length $c^{(2)}_{od}$, the o–d distance in G with the edge $\{o, d\}$ deleted. So $s_{od} + p$ in (3) can be replaced by $c^{(2)}_{od} + p$, or by $h_{od} + p$ when the second group is not retained.

4.2. One-transfer menus

The direct menu charges every transfer journey a fixed fallback, so it cannot reward good transfers. The *shortest-path-leg menu* (SL) gives a transfer at $j \notin \{o, d\}$ the optimistic cost $\hat{c}_{odj} = s_{oj} + s_{jd} + p$, with street distances for the legs. A variable u_{odj} is offered whenever this cost is below

$$\rho^{SL}_{od} = \min\{h_{od}, s_{od} + 2p\}, \tag{9}$$

and it requires both endpoints of each leg on a selected route:

$$u_{odj} \leq \sum_{r\supseteq\{o,j\}} z_r, \qquad u_{odj} \leq \sum_{r\supseteq\{j,d\}} z_r. \tag{10}$$

The menu row becomes $\sum_m \theta_{od,m} + \sum_j u_{odj} \leq 1$, and the objective subtracts the saving $w_{od}(\rho^{SL}_{od} - \hat{c}_{odj})u_{odj}$.

SL underprices legs that a route rides with a detour. The *path-leg menu* (PL) charges actual leg lengths by representing both legs with path groups, built for every pair that may form a leg, including pairs without demand. Route coefficients still stay on nodes and path groups, as pricing and box enumeration require. Availability variables satisfy

$$0 \le \gamma_{ab,m} \le 1, \qquad \gamma_{ab,m} \le \sum_{r \in \mathcal{R}} a_{r,ab,m} z_r. \tag{11}$$

A direct option satisfies $\theta_{od,m} \le \gamma_{od,m}$. An option transferring at j with groups m and n costs $c_{oj,m} + c_{jd,n} + p$ and satisfies $u_{odjmn} \le \gamma_{oj,m}$ and $u_{odjmn} \le \gamma_{jd,n}$. The options of a pair sum to at most one. The fallback

$$\rho^{PL}_{od} = \min \left\{ h_{od},\ s_{od} + 2p,\ \min_{j \notin \{o,d\}} \{h_{oj} + s_{jd} + p,\ s_{oj} + h_{jd} + p\} \right\} \tag{12}$$

gains terms for one-transfer journeys with an omitted leg. For a network R, the *model cost* $f(R)$ of a menu model is its optimal value with z fixed to $\mathbf{1}_R$.

Proposition 1 (Validity of the passenger menus)**.** *For every feasible network R, each menu model above has a feasible solution with $z = \mathbf{1}_R$ and value at most* $\mathrm{TTT}(R)$*. Hence $f(R) \le \mathrm{TTT}(R)$, and the integer and LP optima of the model are lower bounds on the optimum of* (1) *in TTT.*

Proof. Fix a minimum-time journey for each pair. A direct journey in a retained group is available, since a selected route covers the group. Any other direct journey has length at least h_{od}. A journey with a transfer costs at least $s_{od} + p$ (for the direct-edge strengthening, use the case split after (3)), and one with at least two transfers costs at least $s_{od} + 2p$. In SL, a one-transfer journey at j costs at least $\hat{c}_{odj}$ and satisfies (10), so its option is offered or the fallback does not exceed its cost. In PL, a one-transfer journey with retained legs has an available option at its actual cost. If a leg is omitted, it is at least as long as its h value, so (12) does not exceed the journey cost. Take $z = \mathbf{1}_R$, set the availabilities of covered groups to one, and choose for each pair the option of its journey or the fallback. This solution satisfies the menu rows and has value at most $\mathrm{TTT}(R)$. □

4.3. Truncation and refinement

Full lists grow quickly with the instance, so we truncate them by the number K of distinct lengths and a per-pair path cap P. Section 8.4 measures the cost. The cap P keeps the first group, drops trailing groups and lowers h_{ab} to the first omitted length. A safety cap stops generation at 5,000 paths per pair. If it splits a tie group, h_{ab} is the tied length. The PL implementation also refines menus where they underestimate an evaluated network. It keeps a depth per endpoint pair and raises selected depths when more groups can explain the observed passenger cost. Depths only grow, so refinement is finite. Every truncation and depth map gives a valid model, because Proposition 1 needs only a retained prefix of groups and a valid omitted-path threshold. Refinement need not make every network exact, because the fallback $s_{od} + 2p$ may stay below a journey with two or more transfers. Exact search therefore also evaluates and excludes networks (Section 7).

5. Pricing and lower-bound certificates

Section 4 leaves each route with a prize on the nodes and path groups it contains. A group prize depends on a contiguous subpath, so the pricer must recognize listed paths while it extends a route. This section does that and turns every pricing call, finished or not, into a valid master bound.

5.1. Route prizes

Orient the coverage and availability rows of a restricted master so that route coefficients and dual multipliers are nonnegative. Let μ be the free dual of (5), λ_v the node-coverage duals and π_g the group duals. For the D and PL menus, a route has reduced cost

$$\bar{c}_r = -\mu - \Pi(r), \qquad \Pi(r) = \sum_{v \in r} \lambda_v + \sum_{g:\, a_{r,g}=1} \pi_g. \tag{13}$$

Pricing maximizes the prize $\Pi(r)$ over elementary paths with $\ell_{\min} \le |r| \le \ell_{\max}$. A group earns its prize once, however many recognized patterns it contains. For SL the prize also contains nonnegative pair prizes σ_{ab} for routes containing both endpoints, and pricing handles them.

5.2. Recognizing paths while extending a route

Covering a group means containing one of its paths, which is a pattern-matching question. An Aho–Corasick automaton over the node sequences of retained paths, in both directions, recognizes every pattern that ends at the current stop, including those reached through suffix links. A state represents the relevant suffix of the partial route, so the partial path itself is never stored. With q an automaton state, v the last node, ℓ the number of visited stops, additive node weights α_v and automaton-output weights $\beta(q)$, the layered recurrence is

$$\Phi_{\ell+1}(q', v') = \max_{(q,v):\, \{v,v'\} \in E,\ \mathrm{next}(q,v')=q'} \{\Phi_\ell(q, v) + \alpha_{v'} + \beta(q')\}, \tag{14}$$

with initial states at every node and terminal layers $\ell_{\min} \le \ell \le \ell_{\max}$. The recurrence does not enforce elementarity. A walk may revisit nodes and collect a group prize repeatedly, so its optimum can overstate the best route prize.

5.3. A computable upper bound on the pricing optimum

Exact elementary pricing by depth-first search did not finish in our ablation (Table 10), so we penalize repeats. Let $n_v(\varpi)$ count visits to node v in a walk ϖ and $n_g(\varpi)$ collections of group g; in a legal route both are at most one. For nonnegative multipliers ν_v and ξ_g, define

$$U(\nu, \xi) = \sum_v \nu_v + \sum_g \xi_g + \max_{\varpi \in \Omega} \left\{ \sum_v (\lambda_v - \nu_v) n_v(\varpi) + \sum_g (\pi_g - \xi_g) n_g(\varpi) \right\}, \tag{15}$$

where Ω contains all legal routes together with bounded-length walks, and the inner maximum is evaluated by (14). On a legal route each multiplier is subtracted at most once and added once, so every computed $U(\nu, \xi)$ bounds $\max_{r \in \mathcal{R}} \Pi(r)$ from above.

Projected subgradient steps on (ν, ξ) then seek a smaller U (Appendix C). When the optimizing walk repeats nodes or groups, we tighten the walk state: bounded suffix memory removes short cycles, and memory strings for detected repeats forbid their closing repeat without removing any elementary route. The string of the current automaton state is a suffix of the walk read so far, and a transition is removed only when it leads to a node of that string, which closes a repeated node. Every elementary route therefore keeps all its transitions and group counts, and U remains an upper bound whatever strings are stored; the checker only guards the string data (node range). Elementary routes found on the way become columns and bound the best prize from below. Enumeration with dynamic-programming completion bounds can close the remaining gap, but a call may also stop with a certified U. For SL, giving half of each σ_{ab} to each endpoint yields a safe additive bound, and candidate routes are scored with the exact pair prizes.

5.4. From pricing bounds to master bounds

Branching forces and bans routes, adds rows and fixes menu bounds, so we state the bound for a generic node master,

$$\min\ C_0 + c^\top x \quad \text{s.t.} \quad Bx + Az \geq b, \quad \underline{x} \leq x \leq \overline{x}, \quad 0 \leq z \leq 1, \tag{16}$$

where x collects the non-route (menu) columns. Equality rows, among them (5), have free duals, and the rows include the branching rows of Section 7.1. Let F be the forced routes ($z_r = 1$). A legal route is *free* if it is neither forced nor banned. For a dual vector y with the correct signs, route r has reduced cost $\bar{c}_r(y) = -a_r^\top y = -\mu - \Pi_y(r)$ and menu column ι has $\bar{c}_\iota(y) = c_\iota - B_\iota^\top y$. Let

$$\Lambda(y, \delta) = C_0 + b^\top y + \sum_\iota \min\{\bar{c}_\iota(y)\underline{x}_\iota,\ \bar{c}_\iota(y)\overline{x}_\iota\} + \sum_{r \in F} \bar{c}_r(y) + (S - |F|)\,\delta, \tag{17}$$

and call $\delta \in \mathbb{R}$ *admissible* for y if $\bar{c}_r(y) \geq \delta$ for every free route.

Proposition 2 (A bound from any dual point)**.** *If y is sign-feasible and δ is admissible for y, then $\Lambda(y, \delta)$ is at most the LP optimum of (16) and hence at most $f(R)$ for every network R of the node.*

Proof. For a feasible (x, z) the dual signs give $C_0 + c^\top x \geq C_0 + b^\top y + \sum_\iota \bar{c}_\iota(y)x_\iota + \sum_r \bar{c}_r(y)z_r$. Each menu term is at least $\min\{\bar{c}_\iota\underline{x}_\iota, \bar{c}_\iota\overline{x}_\iota\}$, forced routes have $z_r = 1$, banned routes $z_r = 0$, and for the free routes $\sum z_r = S - |F|$ by (5) and $z \geq 0$, so $\sum_{\text{free}} \bar{c}_r z_r \geq (S - |F|)\delta$. □

This is the Lagrangian bound of column generation (Lübbecke and Desrosiers, 2005, eqs. (5) and (16)), with the route-count row in the role of the bound on the number of selected columns, extended by the bounded menu columns and evaluated at any sign-feasible dual point; Vanderbeck and Wolsey (1996) also use lower bounds from pricing to reduce tailing off. Because (5) is an equality, its dual μ is free and $\sum_{r\ \text{free}} z_r = S - |F|$ holds exactly, so δ may have either sign; an inequality $\sum_r z_r \leq S$ would admit only $\mu \leq 0$ and $\delta \leq 0$. If $U \geq \Pi_y(r)$ for every free route, then $\delta = -\mu - U$ is admissible for y. What is new is the computable U, which keeps the bound valid when pricing stops early. We keep the best bound across iterations. The proof does not use $z \leq 1$. Route upper bounds matter only for columns that pricing omits (Appendix A) and for the finite pools of Section 7.3.

Section 6 starts from dual points at which no free route prices out. With $\delta = -\mu - U$ the terms in μ cancel ($S\mu$ in $b^\top y$, $-S\mu$ in the route terms). So replacing μ by $-U$ loses nothing and gives every free route reduced cost $U - \Pi_y(r) \geq 0$. We call such a dual point a *centre*.

We call a bound *certified* when a stored certificate backs it and separate checking code re-verifies it. The checkers rebuild every master from the instance and the stored decisions. A *pricing certificate* stores the menu parameters, y, (ν, ξ) and the memory strings. The checker enforces dual signs, recomputes U by one dynamic program and then recomputes (17). Section 7.8 gives the certificates of the later constructions and the trusted base.

6. Complete route pools

6.1. Local dual optimization

Column generation tails off near a good dual solution. There, only routes with small reduced cost can price out, and the pricing automaton can list them all, so we finish the LP by enumeration. This works for the D and PL menus, whose route coefficients involve only the route count, nodes and path groups. Stabilization guides the dual sequence toward a good centre (Appendix C).

Let y^c be a centre, so that $\bar{c}_r(y^c) \geq 0$ for all free routes, and let $\varepsilon_i \geq 0$ be radii on the route-count, coverage and group rows. Class rows (Section 7.1) are not boxed. With $a_{ir} \in \{0,1\}$ the entries of A in the orientation of Section 5, every route without a class-row coefficient satisfies, in the box $|y_i - y_i^c| \leq \varepsilon_i$,

$$\bar{c}_r(y) \geq \bar{c}_r(y^c) - \sum_i a_{ir}\varepsilon_i, \tag{18}$$

so only routes in

$$\mathcal{R}_{\text{box}} = \left\{ r \text{ free} : \ \bar{c}_r(y^c) \leq \sum_i a_{ir}\varepsilon_i \right\} \tag{19}$$

can have negative reduced cost inside the box. This is a pricing threshold with modified prizes: add ε_v to node v's prize and ε_g to group g's, and lower the prize threshold by the route-count radius. Enumeration collects every elementary route meeting it; one interrupted by a size or time limit gives no complete pool. Let $\mathcal{C}$ be the free members of the node's class rows, explicit columns of the node master.

Proposition 3 (Certification inside a box)**.** *Suppose* (19) *has been enumerated completely, and let y be sign-feasible in the box. Then every free route outside $\mathcal{R}_{\text{box}} \cup \mathcal{C}$ has $\bar{c}_r(y) \geq 0$, and $\delta = \min\{0, \min_{r \in \mathcal{R}_{\text{box}} \cup \mathcal{C}} \bar{c}_r(y)\}$ is admissible for y in Proposition 2.*

Proof. A free route outside $\mathcal{R}_{\text{box}} \cup \mathcal{C}$ has no class-row coefficient, so (18) and (19) give $\bar{c}_r(y) \geq 0$. Every free route therefore has $\bar{c}_r(y) \geq \delta$. □

We optimize over the box by the BOXSTEP method of Marsten et al. (1975): one artificial column per boxed row side, priced at the box edge, confines the dual of the LP over $\mathcal{R}_{\text{box}} \cup \mathcal{C}$ to the box. Its optimal value is a certified bound. If the artificial columns vanish, the box LP solution is feasible for the full LP at that value, so it is the full LP optimum. Otherwise the route-count dual moves to $-U$ with $U = -\mu + \max\{0, -\min_{r \in \mathcal{R}_{\text{box}} \cup \mathcal{C}} \bar{c}_r\}$, giving the next centre, and the radii of binding rows grow. We keep the strongest bound of the chain.

6.2. Complete pools for an improvement cutoff

The search of Section 7 needs only the routes that could appear in an improving network. With a model-cost cutoff in place of a box, the same threshold idea lists all of them. Let y^c be a centre of the node master (route-count dual $-U$) and $\Lambda_c = \Lambda(y^c, 0)$, which includes the full reduced costs of the forced routes and the duals of the branching rows. By the proof of Proposition 2, every network R of the node satisfies

$$f(R) \geq \Lambda_c + \sum_{r \in R \text{ free}} \bar{c}_r(y^c), \qquad \bar{c}_r(y^c) = U - \Pi_{y^c}(r) \geq 0. \tag{20}$$

Proposition 4 (A complete improving-route pool)**.** *For a cutoff τ, every network of the node with $f(R) \leq \tau$ uses only free routes in*

$$\mathcal{R}_\tau = \{ r \text{ free} : \ \bar{c}_r(y^c) \leq \tau - \Lambda_c \}. \tag{21}$$

Proof. All free reduced costs are nonnegative, so a selected route with $\bar{c}_r(y^c) > \tau - \Lambda_c$ alone makes the right-hand side of (20) exceed τ. □

The prize threshold $\Pi_{y^c}(r) \geq U - (\tau - \Lambda_c)$ enumerates this pool. This is the reduced-cost route enumeration of Baldacci et al. (2008), with our pricer as the enumerator.

7. Exact search with passenger evaluation

Root bounds stop short of our networks, and the coverage model underestimates TTT even at integer route selections. The search therefore branches in ways pricing can respect and evaluates every integer point. It keeps an incumbent R^{inc} of true cost T^{inc} and proves that no network has $\text{TTT}(R) \le \tau = T^{\text{inc}} - 10$ (Algorithm 1).

7.1. Dichotomies

Branching must be compatible with pricing (Barnhart et al., 1998). Route branching is compatible, since pricing skips a banned route, but each split removes only one route. Group and class dichotomies act on many routes at once. A node is defined by its decisions; its network set $\mathcal{N}$ holds the feasible networks that satisfy them all. The dichotomies are:

- *Route* r (a free route): the in-child forces $z_r = 1$; the out-child bans r from pricing and enumeration.
- *Path group* g: the required child adds the row $\sum_{r:\, a_{r,g}=1} z_r \ge 1$ over all legal routes, whose dual enters the prize of every covering route. The forbidden child bans every route covering g (prize $-\infty$) and fixes the menu variable of g to zero.
- *Route class* (G_0, κ), with G_0 a set of groups and κ an integer threshold, $1 \le \kappa \le |G_0|$: its members $\mathcal{C}(G_0, \kappa)$, all legal routes covering at least κ groups of G_0, are enumerated completely when the dichotomy is created. The in-child adds the row $\sum_{r \in \mathcal{C}(G_0,\kappa)} z_r \ge 1$ with every member as an explicit column; the out-child bans all members.

Lemma 1 (Dichotomies partition)**.** *For each dichotomy, the network sets of the two children partition $\mathcal{N}$, and every network of a child has the same model cost $f(R)$ in the child as in the parent.*

Proof. A network contains r or not; it contains a route covering g or not; it contains a member of $\mathcal{C}(G_0, \kappa)$ or not. In each case the added row or bans hold exactly for the networks on that side. The added rows involve only z, and in the forbidden-group child the menu variable of g is already zero by (8) or (11), so $f(R)$ is unchanged. □

Candidates are fractional free routes, undecided groups with fractional menu value, and classes built from the groups of a fractional route, with κ a fraction of $|G_0|$. A few column-generation rounds over a preview pool predict each candidate's subtree size; the best is chosen. A class not enumerated within 3,000 routes or 30 seconds leaves only route and group candidates at that node. No route or group is decided twice on a root–leaf path. The current version of the closure run also excludes classes already decided on the path (Appendix B). The completed Mumford0 trees of Section 8.5 predate this change, which affects only progress (Theorem 1).

7.2. Processing a node

A node must close with a bound above τ, yield a network to evaluate, or be split. It inherits its parent's centre, valid because its free routes are a subset of the parent's (new rows get dual zero). Box steps follow: complete enumeration of $\mathcal{R}_{\text{box}}$, the box LP, the bound of Propositions 2 and 3, and growth of the radii of binding rows. Each completed box step certifies a bound, so the node may stop after any step and keep its best certified bound. It is *pruned* when a step's bound exceeds τ; that step's dual and U form its leaf certificate, a threshold-enumeration certificate. The loop also stops when the box is inactive (the node LP is solved) or at a limit of Section 7.6.

Algorithm 1 Certified search over an implicit route universe

Require: Incumbent R^{inc}, cutoff $\tau = \mathrm{TTT}(R^{\mathrm{inc}}) - 10$, root centre, initial menus

Open list $\leftarrow$ {root}
while the open list is nonempty **do**
    Select a node: best-first (closure run: deepest-first while at least 20,000 nodes are open)
    Box steps with complete pools; bound Λ (Props. 2, 3)
    if $\Lambda > \tau$, the node is infeasible, or its only network is an excluded one **then**
        Store the leaf certificate and close the node
    else if the node LP is integral at a network R not yet excluded **then**
        Evaluate R; exclude it or refine depths; if $\mathrm{TTT}(R) \le \tau$, set $\tau \leftarrow \mathrm{TTT}(R) - 10$
    else if $\Gamma \le 4{,}000$ and $|\mathcal{R}_{\tau+1}| \le 1.5 \times 10^6$ and the finisher closes the node **then**
        Store the finisher certificate and close the node
    else
        Choose a route, group or class dichotomy by previews (at an excluded integral point: a free route of it); push both children
    end if
    Re-queue evaluated or stalled nodes; park nodes that cannot be closed or branched
end while
Verify the complete tree (zero open and zero parked nodes) and return R^{inc}

Model and true cost can differ at an integral node LP, so its network R is evaluated. If R is infeasible or $\mathrm{TTT}(R) \le \tau$, it is excluded, and a feasible such R lowers τ to $\mathrm{TTT}(R) - 10$. Otherwise we raise the depths of pairs whose menu cost underestimates R until $f(R) = \mathrm{TTT}(R)$ or no such pair is left. R is excluded if $f(R) \le \tau$ still holds, or if no depth could be raised although the node was solved under the current depth map and cutoff. In every case the node is re-queued. Route dichotomies on its free routes separate an excluded network: the leaf forcing all its S routes holds it alone and is closed by its stored evaluation. This is the canonical cut $\sum_{r \in R} z_r \le S - 1$ (Balas and Jeroslow, 1972) implemented by branching, which avoids unpriced no-good rows in the master. A leaf bound holds for f under that leaf's depth map, hence for TTT (Proposition 1), so nodes may use different maps.

Three devices save time; each yields a certificate of Section 7.8. (i) A child whose preview value exceeds τ is closed at its parent. A threshold enumeration at $U^\star = -\mu + (\Lambda(y, 0) - \tau')/(S - |F|)$ with $\tau' = \tau + 0.02$, which gives $\Lambda(y, -\mu - U^\star) = \tau'$, certifies its preview dual; up to two repair rounds add the free routes found above $U^\star$. (ii) Removal children start from their preview dual with an exact U. (iii) An unpruned node with a fractional LP point and a centre certified by its own enumeration goes to the finisher of Section 7.3 when $0 < \Gamma := \tau - \Lambda_c$ and Γ and its pool $\mathcal{R}_{\tau+1}$ are within the limits of Algorithm 1. Stalled and capped nodes qualify. Appendix B lists the parameter values of Algorithm 1 and the limits of the runs.

7.3. The finite-pool finisher

Box steps re-enumerate a pool at every step. By Proposition 4, the cutoff pool contains the free routes of every network with $f(R) \le \tau$. It shrinks with the gap, so at a small gap it can be enumerated once. Over this finite, explicit pool, $z_r \le 1$ and cuts valid for integral points become usable. Let a node have centre y^c, bound Λ_c and gap $\Gamma = \tau - \Lambda_c > 0$, and let $\mathcal{R}_{\tau+1} = \{r \text{ free} : \bar{c}_r(y^c) \le \Gamma + 1\}$ be enumerated completely. By (20), a network of the node with a free route outside this pool has $f(R) > \tau + 1$. It therefore suffices to show that no network with free routes in $\mathcal{R}_{\tau+1}$ has $f(R) \le \tau$.

The finisher does this by depth-first branch-and-bound with column generation over the pool, branching on routes (in/out) and groups (required/banned within the pool). Its nodes carry the node's rows (plus rows for groups required inside the finisher), the menu columns, the forced routes and the pool routes in $[0,1]$. At the duals y of a finisher LP, let F be the routes forced at that finisher node and $\mathcal{R}_y$ the pool routes free there. Then

$$\Lambda^{(1)} = C_0 + b^\top y + \sum_\iota \min\{\bar{c}_\iota \underline{x}_\iota, \bar{c}_\iota \overline{x}_\iota\} + \sum_{r\in F} \bar{c}_r + \sum_{r\in\mathcal{R}_y} \min\{0, \bar{c}_r\}, \tag{22}$$

$$\Lambda^{(2)} = C_0 + b^\top y + \sum_\iota \min\{\bar{c}_\iota \underline{x}_\iota, \bar{c}_\iota \overline{x}_\iota\} + \sum_{r\in F} \bar{c}_r + (S - |F|)\min\{0, \min_{r\in\mathcal{R}_y} \bar{c}_r\} \tag{23}$$

are lower bounds on $f(R)$ for every network of the finisher node: $\Lambda^{(2)}$ is Proposition 2 on the pool, and $\Lambda^{(1)}$ also uses $z_r \le 1$. A finisher node is a leaf when $\max\{\Lambda^{(1)}, \Lambda^{(2)}\} > \tau$; its dual is stored. A finisher LP integral at a network of model cost at most τ stops the finisher; the main tree evaluates that network, or branches if it is already excluded. The finisher closes a node only when its whole tree is closed.

Over an explicit pool, cuts valid for integral points can tighten the finisher LP. The closure run's finisher adds $\{0, \frac{1}{2}\}$-Chvátal–Gomory cuts at its root (Caprara and Fischetti, 1996). Write $\chi = (x, z)$ for the finisher node's variables. A cut h is given by a multiset I_h of oriented node rows $\hat{a}_i^\top \chi \le \hat{b}_i$ with integer data and a multiset J_h of variables whose upper bound 1 is added, each occurrence with multiplier $\frac{1}{2}$ (a rank-1 Chvátal–Gomory cut). With $\mathbf{m}_h$ the vector of multiplicities in J_h, it reads

$$\Big\lfloor \tfrac{1}{2}\Big(\sum_{i\in I_h} \hat{a}_i + \mathbf{m}_h\Big)\Big\rfloor^\top \chi \le \Big\lfloor \tfrac{1}{2}\Big(\sum_{i\in I_h} \hat{b}_i + |J_h|\Big)\Big\rfloor, \tag{24}$$

rounded componentwise. For integral $\chi \ge 0$ satisfying the rows of I_h and the bound 1 on J_h, the left side is an integer at most $\frac{1}{2}(\sum_i \hat{a}_i + \mathbf{m}_h)^\top \chi \le \frac{1}{2}(\sum_i \hat{b}_i + |J_h|)$, so (24) holds for any (I_h, J_h); no parity argument is needed. Certificates store (I_h, J_h), and the verifier recomputes each cut for its own menu and pool. The cuts serve the finisher only if every network has an integral menu solution of value $f(R)$; Lemma 2 supplies it.

Fix $z \in \{0,1\}^{\mathcal{R}}$. The *PL menu LP* has availability columns γ_g with cost at most zero, $\gamma_g \le N_g := \sum_r a_{r,g} z_r$ and bounds $[0, \overline{\gamma}_g]$, and option columns x_ι of any cost with $x_\iota \le \gamma_g$ for each group g of the option, bounds $[0, \overline{x}_\iota]$ and $\sum_{\iota\in O_{od}} x_\iota \le 1$ for each pair; all upper bounds lie in $\{0,1\}$. Let $\gamma_g^* = \min\{\overline{\gamma}_g, N_g\}$, call an option *available* if $\overline{x}_\iota = 1$ and $\gamma_g^* = 1$ for all its groups, and let $x_\iota^* = 1$ for one cheapest available option of each pair if its cost is negative, and $x_\iota^* = 0$ for all other options.

Lemma 2 (Integral menus)**.** *For every $z \in \{0,1\}^{\mathcal{R}}$, the point (γ^*, x^*) is an integral optimal solution of the PL menu LP.*

Proof. Since N_g is a nonnegative integer and $\overline{\gamma}_g \in \{0,1\}$, γ^* is integral. For a feasible (γ, x), $\gamma \le \gamma^*$, so (γ^*, x) is feasible and no costlier, because availability costs are nonpositive. With γ^* fixed the problem decomposes by pair: minimize $\sum_\iota c_\iota x_\iota$ subject to $\sum_\iota x_\iota \le 1$ and $0 \le x_\iota \le \hat{x}_\iota := \min\{\overline{x}_\iota, \min_g \gamma_g^*\} \in \{0,1\}$. With $c_{\min} = \min\{0, \min_{\iota:\hat{x}_\iota=1} c_\iota\}$, $\sum_\iota c_\iota x_\iota \ge \sum_\iota \min\{c_\iota, 0\} x_\iota \ge c_{\min} \sum_\iota x_\iota \ge c_{\min}$, and x^* attains $c_{\min}$. □

The direct menu is the special case without availability columns. Hence every network R of the finisher node has an integral point $(\mathbf{1}_R, x^*)$ of value $f(R)$ that satisfies all node rows and every cut (24). By Proposition 2 applied to the master with cut rows (whose duals are nonpositive), $\Lambda^{(1)}$ and $\Lambda^{(2)}$ remain lower bounds on $f(R)$. The lemma does not cover SL, and SL never meets the finisher: every search tree of Section 8 uses the PL menu. The verifier checks the lemma's cost premises

(every menu cost of the finisher's master is nonpositive); the menu structure is that of the shared master builder (Appendix B reports a separate check of the cuts).

Reduced-cost fixing at finisher nodes is implemented but unused in reported certificates and the closure run. On the pools of two Mumford0 subproblems we tested dual-path fixing (Yamagishi et al., 2026). Intermediate dual iterates fixed no route beyond the 36–45% of the pool fixed by the final dual. Twenty fix-and-resolve passes over alternative optimal duals reached 59–69%, yet the finisher trees were essentially unchanged.

7.4. Correctness and finite convergence

Node order, dichotomy choice, limits and parking are heuristics, so correctness must not depend on them. Theorem 1(a) states what a completed tree proves; (b) proves that the search finishes when every node is processed exactly, without resource limits.

For (a), consider a tree whose root is the set of feasible networks containing a base set R^{base} of routes ($R^{\mathrm{base}} = \emptyset$ for a global tree) and whose internal nodes are split by the dichotomies of Section 7.1. Let τ_0 be the start cutoff and $\mathcal{E}$ a list of evaluated networks, each listed at a cutoff $\tau(\tilde{R})$. Call $\tilde{R}$ *improving* if it is feasible and $\mathrm{TTT}(\tilde{R}) \le \tau(\tilde{R})$, and let

$$\tau_f = \min\bigl\{\tau_0,\ \min\{\mathrm{TTT}(\tilde{R}) - 10 :\ \tilde{R} \in \mathcal{E} \text{ improving}\}\bigr\}. \tag{25}$$

A leaf is *settled* if it is infeasible, consists of a single network of $\mathcal{E}$, or is closed at a recorded cutoff τ_{leaf} by a certified bound on f greater than τ_{leaf} (a pruned leaf, or a node whose finisher tree is closed). The tree is *settled* if every leaf is settled and no cutoff τ_{leaf} or $\tau(\tilde{R})$ is below τ_f.

For (b), let d be the current depth map and $\mathcal{R}^{\mathrm{f}}_N$ the set of free routes of a node N, a finite set. The *node LP* P_N is the LP relaxation of the node master (16) under d with every route of $\mathcal{R}^{\mathrm{f}}_N$ as a column, $z_r = 1$ for forced routes, $z_r = 0$ for banned routes and $z_r \ge 0$ for free routes; as in Proposition 2, the bounds $z_r \le 1$ of free routes are dropped. An *exact processing* of N at the current cutoff τ does the following.

(P1) It solves P_N exactly, for instance by the simplex method over the enumerated set $\mathcal{R}^{\mathrm{f}}_N$.

(P2) If P_N is infeasible or its optimum exceeds τ, it closes N.

(P3) Otherwise let (x^*, z^*) be the optimal solution returned. If $z^*_r \notin \{0, 1\}$ for some route r, it splits N by a dichotomy whose object is not yet decided on the root–N path; class members are enumerated completely.

(P4) If z^* is binary, let $R = \{r : z^*_r = 1\}$. If R is not excluded, it evaluates R. If R is infeasible or $\mathrm{TTT}(R) \le \tau$, R is excluded and, if R is feasible, R becomes the incumbent and $\tau \leftarrow \mathrm{TTT}(R) - 10$. Otherwise it refines the depths to a map $d' \ge d$ and excludes R if $f_{d'}(R) \le \tau$, where $f_{d'}$ is the model cost under d'. In both cases N is re-queued. If R was already excluded, it closes N when all routes of R are forced, and otherwise splits N by the route dichotomy on a route of R that is not forced.

Nodes are processed one at a time, in any order. There are no limits, no stall or hold rule, no parking and none of the devices (i)–(iii) of Section 7.2.

Theorem 1 (Sound trees and finite convergence)**.** *(a) If the tree is settled, every feasible network R of the root has $\mathrm{TTT}(R) > \tau_f$. If moreover $\tau_0 = T^{\mathrm{inc}} - 10$ with R^{inc} a network of the root, then $\mathrm{TTT}(R) \ge \tau_f + 10$, and this value is attained by the incumbent or by the best improving listed network, which is therefore optimal over the root.*

(b) Algorithm 1 with exact processing, an exact evaluator and exact arithmetic stops after finitely many processings with a settled tree, and its final cutoff is τ_f.

Proof. (a) By Lemma 1 and induction over the tree, the leaves partition the root's networks. A network in a closed leaf has $\mathrm{TTT}(R) \geq f(R) > \tau_{\mathrm{leaf}} \geq \tau_f$ by Proposition 1, for the depth map of that leaf (for a finished node, every network either lies in a closed finisher leaf or uses a route outside the finisher's pool and has $f(R) > \tau_{\mathrm{leaf}} + 1$). A feasible network $\tilde{R}$ of a single-network leaf has $\mathrm{TTT}(\tilde{R}) \geq \tau_f + 10$ if it is improving, by (25), and $\mathrm{TTT}(\tilde{R}) > \tau(\tilde{R}) \geq \tau_f$ otherwise. If $\tau_0 = T^{\mathrm{inc}} - 10$, every cutoff, and hence τ_f, lies on the lattice $10\mathbb{Z}$, so $\mathrm{TTT}(R) > \tau_f$ means $\mathrm{TTT}(R) \geq \tau_f + 10$. If some listed network is improving, the one of least TTT attains $\tau_f + 10$; otherwise $\tau_f = \tau_0$ and the incumbent attains $\tau_0 + 10 = T^{\mathrm{inc}}$.

(b) *One processing.* Every processing of a node ends in one of three ways: the node is closed as a settled leaf; it is split by a dichotomy on an object not yet decided on its root–node path; or a binary node-LP solution gives an evaluated set of S routes (its union may be disconnected) that is excluded for the first time or strictly raises the depth map; cases (i)–(iv) below show this. Let N be processed at cutoff τ under the map d, let F be its forced routes and Λ^* the optimum of P_N. Every network R of N gives a feasible point of P_N: $z = \mathbf{1}_R$ with the menu solution of Proposition 1, which satisfies the branching rows as in the proof of Lemma 1. Call y *dual feasible* if it is sign-feasible and $\bar{c}_r(y) \geq 0$ for every $r \in \mathcal{R}_N^{\mathrm{f}}$. For such y, $\delta = 0$ is admissible in Proposition 2, and $\Lambda(y, 0)$ is the dual objective of P_N at y once the multipliers of the variable bounds are chosen optimally. The point $y = 0$ is dual feasible.

(i) If P_N is infeasible, LP duality and the dual feasible point $y = 0$ make the dual objective unbounded, so some dual feasible y has $\Lambda(y, 0) > \tau$. Then N holds no network and is closed as a pruned leaf. Its threshold-enumeration certificate is (y, U) with $U = -\mu$, since $\bar{c}_r(y) \geq 0$ means $\Pi_y(r) \leq U$ on $\mathcal{R}_N^{\mathrm{f}}$. This case includes every node with contradictory decisions, such as more than S forced routes; the implementation closes combinatorial contradictions as infeasible leaves (Section 7.8); (a) admits both.

(ii) If $\Lambda^* > \tau$, an optimal dual y^* is dual feasible and $\Lambda(y^*, 0) = \Lambda^*$ by strong duality, so N is closed in the same way.

(iii) If $\Lambda^* \leq \tau$ and $z_r^* \notin \{0, 1\}$, then r is free, since forced and banned routes are fixed at 1 and 0. A route decided on the root–N path is forced (in) or banned (out) at N, so r is undecided and the route dichotomy on r is available. This covers $z_r^* \geq 2$, which is possible without the bounds $z_r \leq 1$.

(iv) If $\Lambda^* \leq \tau$ and z^* is binary, R consists of S distinct legal routes by (5), covers V by (6), contains F, avoids every banned route and satisfies the branching rows. So R satisfies every decision of N, and it is a network of N unless its union is disconnected. As in Lemma 1, the node master with $z = \mathbf{1}_R$ has optimal value $f_d(R)$, the model value at $z = \mathbf{1}_R$, which is defined for any set of S routes. The point $(x^*, \mathbf{1}_R)$ is feasible there, and $\mathbf{1}_R$ with an optimal menu solution is feasible for P_N, so $f_d(R) = \Lambda^* \leq \tau$. Suppose R was not excluded. If (P4) does not exclude it, then $\mathrm{TTT}(R) > \tau$ and $f_{d'}(R) > \tau \geq f_d(R)$, so $d' \neq d$ and the depth map strictly grows; otherwise R is excluded for the first time. Suppose R was excluded. If $F = R$, every network of N contains the S routes of R and so equals R: N is a leaf whose only possible network is the listed network R, and it holds no network if R is infeasible. If $F \neq R$, a route $r \in R \setminus F$ has $z_r^* = 1$, so it is free and, as in (iii), undecided; the route dichotomy on r is available, and its out-child does not contain R.

Finiteness. Routes, groups (of finitely many simple paths, over all depth maps) and classes (pairs (G_0, κ) with $1 \leq \kappa \leq |G_0|$) form a finite family $\mathcal{O}$. Dichotomies decide only objects not yet decided on the path, so every root–leaf path has length at most $|\mathcal{O}|$ and the tree has fewer than $2^{|\mathcal{O}|+1}$ nodes. A processing that closes or splits a node is that node's last, so there are fewer than $2^{|\mathcal{O}|+1}$ such processings. Every other processing excludes a network for the first time, which happens at most once per set of S legal routes, or strictly raises the depth map, which happens finitely often since depths only grow and are bounded by the number of distinct simple-path lengths of each pair. Each processing is a finite computation: an LP over finitely many columns, a finite enumeration

of class members, one evaluation and finitely many depth raises. Hence the open list empties after finitely many processings.

The final tree. Its internal nodes are split by dichotomies of Section 7.1, and every leaf was closed in (i), (ii) or (iv) as a settled leaf: pruned at the cutoff current when it was processed, or holding at most one network of the list $\mathcal{E}$ of networks evaluated in (iv), each listed at the cutoff of its evaluation. The cutoff decreases only when an improving network $\tilde{R}$ is evaluated, to $\mathrm{TTT}(\tilde{R}) - 10 < \tau(\tilde{R})$, so the final cutoff is τ_f and every recorded cutoff is at least τ_f. Part (a) applies.

Parallel processing. A pruned, infeasible or split result computed in parallel under an older map or a higher cutoff is kept, as (a) and Lemma 1 admit. An integral result is evaluated against the current map, cutoff and excluded list, and the node is re-queued; if nothing changes, the result was stale. With W workers each change of these makes at most W results stale, and they change finitely often, so (b) also holds for parallel exact processing. □

The verifier checks part (a) for a completed tree (Appendix B), deriving τ_f from the listed networks alone and checking every recorded cutoff against it. Part (a) also covers trees without an incumbent: with Λ_{root} the certified root bound, a tree closed at a cutoff $\tau = \Lambda_{\mathrm{root}} + \Delta$ with $\mathcal{E} = \emptyset$ proves $\mathrm{TTT}(R) > \tau$ for every feasible network. The Mumford0 lower bounds of Table 13 are of this kind.

7.5. Lattice cutoffs and strict acceptance

Because $\mathrm{TTT} \in 10\mathbb{Z}$, a cutoff one lattice step below the incumbent suffices (Theorem 1(a)). This needs strict acceptance: a leaf counts only if its certified bound exceeds its cutoff, which excludes $\mathrm{TTT} = \tau_{\mathrm{leaf}}$ itself. A leaf closed under an earlier, larger cutoff stays closed. The solver prunes at a margin of 10^{-6}; the verifier requires a positive margin. Root-certificate checks use a separate absolute tolerance of 10^{-3} TTT, which the data tables subtract from accepted root bounds.

7.6. Resource limits and progress

Theorem 1(b) covers exact processing only. Outside it lie (1) limits on enumeration size and time, node time, box steps, class enumeration and the finisher, and the stall and hold rules of the closure run, after which a node keeps its best certified bound, is closed only if that bound exceeds τ, and is otherwise split, re-queued or parked; (2) *parking*, which sets a node that keeps failing aside until a manual restart; (3) devices (i)–(iii) and the finisher cuts; (4) floating-point arithmetic, with the margins of Section 7.5; and (5) code changes during the closure run, before the last of which a class could be decided twice on a path. Part (a) uses none of this, only the completed tree (the conditions of settledness are checked in floating point, for Tables 13 and 14 the smallest margins also exactly): complementary children at every internal node (Lemma 1 holds whether or not the object was decided before), a stored certificate at every leaf, rechecked at the leaf's own cutoff and depth map, and recorded cutoffs not below τ_f. Limits, rules, node order and code versions decide which tree is built and whether it is completed, not what a completed tree proves. Appendix B gives these rules and changes. A run is complete only with zero open and zero parked nodes and a verified tree.

7.7. Certifying route-exchange neighbourhoods

A global tree at the incumbent's cutoff is expensive (Section 8.6); trees that force most incumbent routes are smaller. Let R^0 be an incumbent and $X \subseteq R^0$ a set of released routes. The neighbourhood $\mathcal{B}(X) = \{R \text{ feasible} : R \supseteq R^0 \setminus X\}$ is the root of a tree in which the routes of $R^0 \setminus X$ are forced; the other $|X|$ routes may be any legal routes, including those of X. We call X *certified* if a verified tree for $\mathcal{B}(X)$ at cutoff $\mathrm{TTT}(R^0) - 10$ proves $\mathrm{TTT}(R) > \mathrm{TTT}(R^0) - 10$, hence $\mathrm{TTT}(R) \geq \mathrm{TTT}(R^0)$, for

every $R \in \mathcal{B}(X)$. Replacing at most k routes is the local-branching neighbourhood $\sum_{r \in R^0}(1-z_r) \le k$ of Fischetti and Lodi (2003, eq. (8)), which we close by trees with forced routes.

Corollary 1 (Covering exchange neighbourhoods)**.** *Let $k \le S$. If every k-subset of R^0 is contained in a certified released set, then every feasible network R with $|R \cap R^0| \ge S - k$ has* $\mathrm{TTT}(R) \ge \mathrm{TTT}(R^0)$.

Proof. $R^0 \setminus R$ has at most k routes; extend it to a k-subset $Y \subseteq R^0$ and choose a certified $X \supseteq Y$. Then $R \supseteq R^0 \setminus Y \supseteq R^0 \setminus X$, so $R \in \mathcal{B}(X)$. □

7.8. Certificates and the trusted base

Besides pricing certificates (Section 5.4), three kinds of certificate back the bounds of box steps and search nodes. A *box chain* (Section 6) stores the centres and radii. The checker re-enumerates each box pool and checks the box constraints, the minimum reduced cost and the link to the next centre. A *threshold-enumeration certificate* stores (y, U) at a search node (Section 7.2). The checker enumerates every free route with $\Pi_y(r) \ge U$, checks that none exceeds $U + 10^{-6}$, checks explicit columns such as class members directly, and evaluates (17) at $\delta = -\mu - (U + 10^{-6})$. A *finisher certificate* stores the node's centre (y, U), from which the checker re-enumerates the finisher's pool, and the finisher tree with one dual per leaf, checked by (22)–(23) over that pool (Section 7.3). *Infeasible* leaves need no certificate; they are proved combinatorially: more than S forced routes, a forced route that is banned, illegal or covers a forbidden group, a class row without a usable member, or a required group that no route can cover. The checker re-derives the reason; infeasibility of a restricted master is never used. All 34 infeasible leaves of our trees are of this kind.

The checkers share the instance loader, the master builder and the compiled automaton core with the solver. These, the k-shortest-path group construction, the box-step module (whose separate function checks box chains), the passenger evaluator and floating-point arithmetic within the stated tolerances form the trusted base. The tree and cut checkers are separate programs that do not import the branching code. We cross-check the completeness of the compiled enumerator by a sampled run of the Lagrangian pricer and by re-pricing a sample of enumerated routes from their column definition. The master builder and Proposition 1 as implemented are tested separately, and the evaluator is validated against published route sets (Section 8.1).

8. Computational study

8.1. Instances, implementation and evidence

Table 1 gives the instance parameters. The optimizer, the evaluator and the certificate checkers share one problem definition. The master models, experiment control and verification are written in Python, the automaton dynamic program and route enumeration in C++. HiGHS 1.15.1 (Huangfu and Hall, 2018) solves all LPs and MIPs. The software was developed and the experiments were run with the assistance of AI coding agents (OpenAI Codex and Anthropic Claude Code).

Unless stated otherwise, runs used a shared Apple M4 with 10 cores and 16 GiB of memory (Python 3.12.13); the three cold Mumford root campaigns ran on it concurrently. The Mandl runs for $S = 3, \dots, 14$, the pricing and box ablations, the expansion cases and the $P = 80$ root campaigns used a shared AMD Ryzen 7 6800H laptop under WSL, and the Mumford0 closure run a server (Section 8.6).

The instances have 1,291 (Mandl) and 3,431,531,559 (Mumford0) legal routes, counted exactly; Knuth's estimator (Knuth, 1975) with 1×10^8 probes gives an estimated 1.0×10^{19}, 1.1×10^{17} and 4.5×10^{18} on Mumford1–3.

Table 1: Benchmark parameters. Route bounds $\ell_{\min}, \ell_{\max}$ count stops; W counts demand in both directions. The Mandl file has 21 street links, while Mumford (2013, Table II) prints 20; sources differ on Mandl's minimum route length (2, 3 or 5 stops, or none).

Instance	$\lvert V\rvert$	$\lvert E\rvert$	S	$\ell_{\min}$	$\ell_{\max}$	W
Mandl	15	21	6	2	8	15,570
Mumford0	30	90	12	2	15	342,160
Mumford1	70	210	15	10	30	1,926,170
Mumford2	110	385	56	10	22	4,847,900
Mumford3	127	425	60	12	25	6,394,950

We validated the evaluator on 279 published route sets. Of the 61 with a printed passenger-time value, 57 agree at the printed precision; the 4 discrepancies are Mandl sets of Hüsselmann et al. (2024), whose demand matrix has $D_{34} = 10$ where the benchmark file has 40; with that entry all their printed metrics reproduce. 63 Mandl sets violate the route-length bounds, and all 120 published Mumford sets are feasible.

Stored networks, the 5 root certificates, the $P = 80$ box chains, the 2 trees of Table 13 and the 46 released sets were re-verified from the data snapshot, the Mandl and expansion trees when produced; in exact arithmetic the smallest margins stay positive (Appendix B). Lower bounds are rounded down, gaps up, and all other values to the nearest unit of the last digit.

8.2. Exact solutions and formulation size on Mandl

Mandl's 1,291 legal routes can all be listed, so we solve it exactly. We solve the coverage MIP (direct $K = 5$ menu plus SL options), evaluate the selected network and add a canonical cut, until the best remaining bound meets the incumbent. Table 2 gives the optima for $S = 3, \ldots, 14$ with routes of 2–8 stops. All twelve equal Vermeir et al. (2021, Table 1) and Gao et al. (2026, Table 6). Vermeir et al. allow at most S lines; requiring exactly S gives the same optimum, because adding a distinct legal route never increases a journey cost. Every optimal route has at least four stops, so the optima also hold under the three-stop minimum of Gao et al. (2026, Table 3). Small S needs many iterations (80 for $S = 4$), large S one. These full-pool times are for comparison with Vermeir et al. and Gao et al., on different hardware (Vermeir et al.: four cores of an Intel Xeon E5-2640 v4 virtual machine; Gao et al.: an Intel i7-11800H laptop with Gurobi 9.5.2).

Algorithm 1 proves the same twelve optima without listing the pool (Table 2). Each global tree starts from the PL $K = 5$ root certificate at the cutoff optimum minus 10; the trees exclude 5 evaluated networks in all, take 38.8 wall seconds in total on the Ryzen laptop, and all 12 pass the verifier.

Full-pool LPs (Table 3) show which strengthening helps. More distinct path lengths raise the direct-menu bound for $S = 4, \ldots, 8$ up to $K = 5$, and $K = 10$ changes nothing. SL helps only for $S = 4$ and $S = 5$.

The menus lose nothing here against the passenger-flow LP without a transfer cut. On the complete pool, the direct coverage LP equals our implementation of the route-slot flow formulation of Gao et al. (2026) with $K = 5$ shortest-path inequalities in all six cases, with 12.4–35.5 times fewer columns (Table 4). The slot formulation needs the complete pool, which on Mumford0 has 3.4×10^9 routes, so we compare it only on Mandl. Gao et al. (2026, Table 9) report root bounds that equal ours for $S = 6$–8 and exceed them for $S = 4, 5$ (at $\lvert M\rvert = 5$, 162,227 against our 162,110 and 160,036 against 159,960). Neither their five-stop minimum nor their counting of shortest paths instead of path groups in $\lvert M\rvert$ explains this: with a five-stop minimum our LP is unchanged, and counting

Table 2: Mandl optima with routes of 2–8 stops. Alg. 1: nodes and wall seconds of the global tree of Algorithm 1, started from the root certificate (root column generation at most 1.4 s). Full pool: exact search over the full route pool. Our times are wall seconds on the Ryzen laptop; Vermeir and Gao: published CPU seconds (Vermeir et al., 2021, Table 1), (Gao et al., 2026, Table 6).

S	Optimal TTT	ATT	Alg. 1 nodes	Alg. 1 (s)	Full pool (s)	Vermeir (s)	Gao (s)
3	169,450	10.8831	29	21.2	1,406.4	13	187
4	163,210	10.4823	11	8.8	2,575.8	297	711
5	160,450	10.3051	1	2.0	924.8	3,317	831
6	158,500	10.1798	1	0.8	222.5	18,856	217
7	157,260	10.1002	1	0.6	32.4	39,753	215
8	156,750	10.0674	1	0.8	12.4	161,902	173
9	156,300	10.0385	1	0.7	14.0	200,872	386
10	155,940	10.0154	1	0.7	3.4	29,296	142
11	155,850	10.0096	3	1.4	3.3	17,054	174
12	155,820	10.0077	1	0.6	8.0	16,320	216
13	155,800	10.0064	1	0.6	3.1	24,883	320
14	155,790	10.0058	1	0.6	3.4	1,019	379

Table 3: Mandl full-pool LP bounds in TTT. D: direct menu with K path groups; SL: shortest-path-leg menu. D with $K = 10$ equals D with $K = 5$ in every row.

S	D, $K = 1$	D, $K = 3$	D, $K = 5$	SL, $K = 5$
4	158,050.00	161,583.33	162,110.00	162,230.00
5	157,280.00	159,420.00	159,960.00	160,051.66
6	156,750.00	158,164.00	158,428.00	158,428.00
7	156,430.00	157,080.00	157,250.00	157,250.00
8	156,180.00	156,570.00	156,715.00	156,715.00
12	155,820.00	155,820.00	155,820.00	155,820.00

Table 4: Mandl formulation comparison at $K = 5$. Both formulations have the LP value shown; the coverage LP has 1,717 columns and 528 rows for every S. Times are single LP solves on the same machine.

S	LP value (TTT)	Slot cols.	Slot rows	Slot (s)	Cover (s)
4	162,110.00	21,324	20,909	3.287	0.042
5	159,960.00	26,269	25,682	4.118	0.026
6	158,428.00	31,214	30,455	4.619	0.028
7	157,250.00	36,159	35,228	3.983	0.023
8	156,715.00	41,104	40,001	4.945	0.024
12	155,820.00	60,884	59,093	6.861	0.020

paths lowers it. Adding their transfer cut (Gao et al., 2026, eq. (42)) to our slot LP reproduces all 15 cells, rounded down, for both minimum lengths, so their roots appear to include it (their Section 4.5 generates it at branching nodes). Our SL menu, without that cut, gives bounds slightly above their $|M| = 5$ values for $S = 4, 5$ (Table 3).

Table 5: Certified bounds and feasible networks (ATT in minutes). L_0: street bound LBpass of Mumford (2013, Table II), rounded down (hence 13.0120 and 24.7452 against the printed 13.0121 and 24.7453). Printed: best values of the genetic algorithm with sub-objectives (GASO) in Wens and Vansteenwegen (2025, Table 4), a table that also compiles earlier results such as those of Mumford (2013), John et al. (2014) and Ahmed et al. (2019); the corresponding networks are not public. Best public: lowest value of a published route set under our evaluator, all of them networks of Wens and Vansteenwegen (2025) that satisfy every benchmark constraint, including connectivity. LB: best certified bound, for Mumford0 from the completed tree of Table 13, otherwise the root bound of Table 8. Our UB: ATT of our network.

Instance	L_0	LB	Printed	Best public	Our UB	Gap (%)
Mumford0	13.0120	13.7641	13.95	13.9925	13.8018	0.28
Mumford1	19.2695	20.9335	21.50	21.5572	21.2057	1.29
Mumford2	22.1689	23.5115	24.22	24.2306	23.9997	2.04
Mumford3	24.7452	25.9912	26.92	26.9657	26.7372	2.79

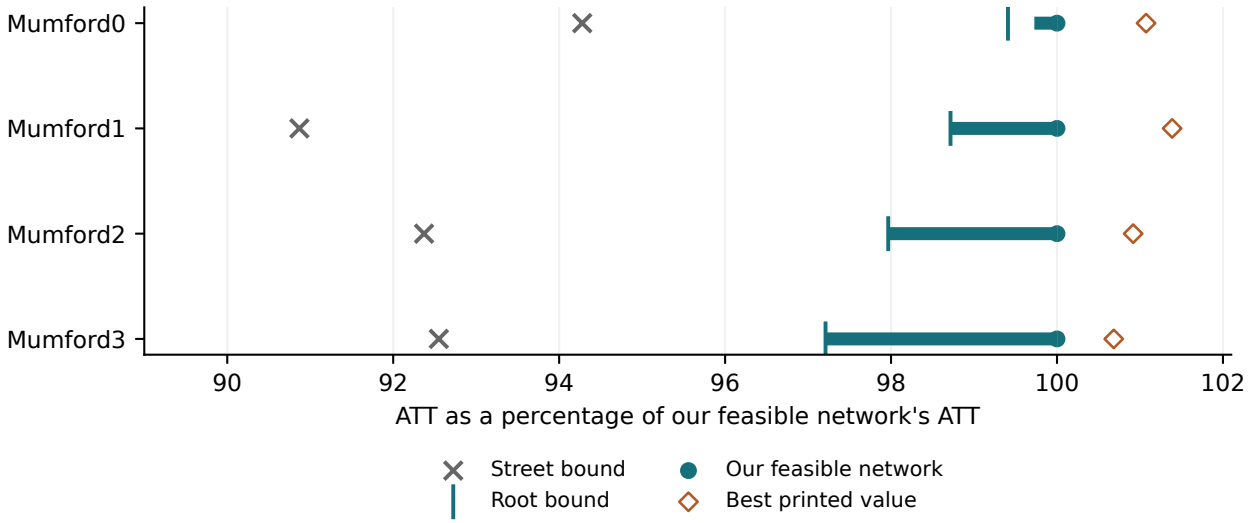


Figure 1: Bounds and values relative to our feasible ATT. Thick segments run from the best certified lower bound to our network; the printed values are those of Wens and Vansteenwegen (2025, Table 4).

8.3. Bounds and feasible networks on Mumford

Table 5 and Figure 1 give the certified bounds and our networks, and Table 6 the transfer shares. The gap $100(\text{UB} - \text{LB})/\text{UB}$ is computed from unrounded values. Our networks, from the separate heuristic of Appendix D, beat the printed values by at least 1.06, 1.36, 0.90 and 0.67%, and no printed value we found for the standard problems lies below them. The printed values are lower than those of the best published networks we could re-evaluate, so this comparison favours the literature.

On Mumford0 the D, SL and PL root LPs at $K = 5$ all give 13.7202, and branching raises the bound to 13.7641 (Section 8.5). Table 8 lists the SL certificate, the trees start from the PL certificate and Table 10 uses the D certificate. Mumford1 uses the uncapped direct menu, Mumford2–3 the direct menu with $P = 80$. The Mumford0 and Mumford1 root bounds are LP optima of their menus (computed): re-solving the restricted master over the stored routes exceeds the certified bound by at most the LP bracket of Table 8, and on Mumford1 the final box step was inactive. More column generation cannot raise them. On Mumford2–3 the $P = 80$ box chains stopped at an incomplete

Table 6: Transfer shares (percent of demand) of our networks and of the best public networks, with the fewest-transfer tie rule. In every network all demand travels with at most one transfer ($d_2 = d_{un} = 0$). For Mandl ($S = 6$) our optimum and the best public set have the same value.

Instance	Network	ATT	d_0	d_1
Mandl	ours	10.1798	97.17	2.83
	best public	10.1798	97.17	2.83
Mumford0	ours	13.8018	92.55	7.45
	best public	13.9925	88.47	11.53
Mumford1	ours	21.2057	72.49	27.51
	best public	21.5572	67.15	32.85
Mumford2	ours	23.9997	75.29	24.71
	best public	24.2306	70.78	29.22
Mumford3	ours	26.7372	70.71	29.29
	best public	26.9657	66.87	33.13

Table 7: Phases of the root campaigns: the cold campaigns and the $P = 80$ campaigns started from their routes; each phase continues from its predecessor. P: per-pair path cap (none on Mumford1). Certified LB in ATT, with six decimals to show the box gains. Time: wall hours of the phase.

Instance	Phase	P	Certified LB	Time (h)
Mumford1	Ordinary CG	–	20.881324	1.02
	Stabilized CG	–	20.933524	1.91
	Dual box	–	20.933531	0.04
Mumford2	Ordinary CG	40	23.262678	0.84
	Stabilized CG	40	23.361758	16.09
	Dual box	40	23.361925	0.08
	Ordinary CG	80	23.365836	4.01
	Stabilized CG	80	23.495173	12.00
	Dual box	80	23.511524	0.10
Mumford3	Ordinary CG	40	25.754755	1.00
	Stabilized CG	40	25.857122	16.00
	Dual box	40	25.858968	0.31
	Ordinary CG	80	25.927657	5.43
	Stabilized CG	80	25.977135	12.01
	Dual box	80	25.991282	0.04

enumeration, so the LP optimum is not established; it lies between the certified bound and the restricted-master LP value recorded by the stabilized phase, 23.5303 and 26.0431 (computed).

The cold Mumford1–3 campaigns (Table 7, Figure 2; settings in Appendix C) took 3.0, 17.0 and 17.3 wall hours. Ordinary column generation gives about 97%, 92% and 91% of the rise from the street bound to the final bound. On Mumford2 and Mumford3 the stabilized phase adds about 0.10 minutes but ends at its 16-hour budget short of the LP optimum; the box phase reaches it in 302 and 1,127 seconds. The $P = 80$ campaigns start from the routes of the cold campaigns and take 16.1 and 17.5 wall hours (plus 3.8 hours of two stopped Mumford3 runs); their box phases end with the box still active.

Table 8: Root certificates behind Table 5, all with $K = 5$. Menu: D direct, SL shortest-path legs; no P cap is shown as a dash. LP optimum: computed, or not established when the box enumeration stopped incomplete. LP bracket: re-solved restricted-master LP value (Mumford2–3: the value recorded by the campaign) minus the certified bound.

Instance	Menu	P	Root LB	LP optimum	LP bracket (TTT)
Mumford0	SL	–	13.7202	computed	1.5×10^{-8}
Mumford1	D	–	20.9335	computed	1.1×10^{-6}
Mumford2	D	80	23.5115	not established	9.1×10^{4}
Mumford3	D	80	25.9912	not established	3.3×10^{5}

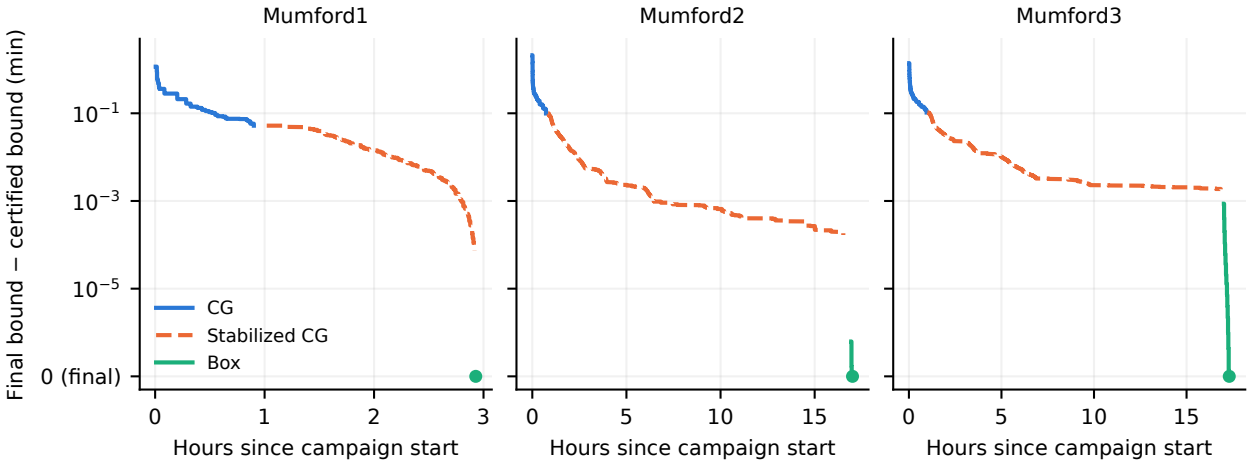


Figure 2: Distance of the certified bound from the campaign's final bound during the cold campaigns of Table 7; the dot marks the end of the box phase.

Table 9: Box step started from the ordinary column-generation (CG) certificate, without the stabilized phase, against the cold campaign (Stab. + box). Bounds in ATT, rounded down, with six decimals as in Table 7. Routes at failed ε: size of the enumeration and the radius at which it stopped incomplete. All box chains were re-verified.

Instance	CG	Box after CG	Routes at failed ε	Stab. + box
Mumford1	20.881324	20.902493	3,471,196 at 1,280	20.933531
Mumford2	23.262678	23.281323	3,314,329 at 320	23.361925
Mumford3	25.754755	25.766298	3,622,916 at 320	25.858968

The stabilized phase looks idle, yet the box step needs it (Table 9). It comes within 10^{-3} minutes of its final value after 1.78, 5.52 and 8.10 hours (phase lengths in Table 7). Started directly from the ordinary column-generation certificate, the box enumeration became incomplete at the next radius, and the bounds stopped at 20.9024, 23.2813 and 25.7662, against 20.9335, 23.3619 and 25.8589 with it.

Table 10 re-prices at the stored certificate duals. On Mumford2 and Mumford3 the full pricer started cold recovers the certified U, and penalties with memory strings alone suffice; dropping the penalties, or both kinds of memory, loses 0.007–0.033 minutes. On Mumford0 and Mumford1 only the pricer state built up during the run recovers U; a cold start ends 0.060 and 0.074 minutes short. Exact elementary pricing by depth-first search did not finish within 30 minutes at any point. Appendix C gives pricing statistics.

Table 10: Pricing-bound ablation at fixed dual points (Mumford0 root certificate of the D menu, whose value equals the SL root; Mumford1 stabilized; Mumford2–3 $P = 40$ box certificates): bound minus certified bound in ATT (0: certified value reproduced). Memory: suffix memory of the walk state; strings: memory strings added for detected repeats. Variants were limited to 600 s, the exact elementary depth-first search to 1,800 s (^{t}: the run reached its limit). A dash marks a certificate without stored pricer state.

Variant	Mumford0	Mumford1	Mumford2	Mumford3
Stored pricer state	0	0	–	–
Full pricer, cold start	-0.0597	-0.0740	0	0
No penalties, memory 5	-0.3250	-0.1552	-0.0292	-0.0223
No penalties, memory 5, strings	-0.0795^t	-0.0702^t	-0.0072^t	-0.0077^t
Penalties, no memory	-0.1083	-0.1152	-0.0330	-0.0235
Penalties, no memory, strings	-0.0665	-0.0732	0	0
Node penalties, memory 5, strings	-0.0665	-0.0680^t	0	0
Depth-first search	-11.9368^t	-16.4959^t	-12.6546^t	-12.7270^t

Table 11: Gap decomposition (26) at our best networks (ATT(R^*): Our UB in Table 5), in minutes, with the menus of the reported bounds (Mumford0: SL root and PL tree, uncapped; Mumford1 uncapped; Mumford2–3 $P = 80$).

Instance	LB	Gap	Menu term	Remainder	Menu share (%)
Mumford0	13.7202	0.0816	0.0021	0.0795	2.6
Mumford0 (tree)	13.7641	0.0377	0.0021	0.0356	5.7
Mumford1	20.9335	0.2722	0.0409	0.2314	15.0
Mumford2	23.5115	0.4882	0.3057	0.1826	62.6
Mumford3	25.9912	0.7459	0.5076	0.2383	68.1

8.4. Where the remaining gap comes from

The root bounds are LP optima of their menus on Mumford0–1 and within 0.019 and 0.052 of them on Mumford2–3 (Table 8), so progress needs stronger menus, search or, on Mumford2–3, a finished LP. To tell which, we fix each best network R^* and split the gap at its model cost $f(R^*)$:

$$\mathrm{ATT}(R^*) - \mathrm{LB} = \big[\mathrm{ATT}(R^*) - f(R^*)/W\big] + \big[f(R^*)/W - \mathrm{LB}\big]. \tag{26}$$

The first term, the menu term, is the menu's underestimate at R^*. The second caps what search that prunes by model-cost bounds can add with these menus, since $\min_R f(R)/W \le f(R^*)/W$.

On Mumford2 the menu term is 0.3057 of the 0.4882-minute gap (62.6%), on Mumford3 0.5076 of 0.7459 (68.1%; Table 11). Such search can therefore add at most 0.1826 and 0.2383 minutes; the rest needs stronger menus, evaluation and exclusion, or a better network. The menu term barely depends on the network: at three earlier networks of ours on Mumford2 and two on Mumford3 it differs by at most 0.010 minutes, and at the best public networks it is larger (0.38 and 0.61 minutes).

Removing the per-pair cap ($K = 5$, network fixed; Table 12) cuts the menu term from 0.3057 to 0.0229 minutes on Mumford2 and from 0.5076 to 0.0226 on Mumford3. The uncapped menus, however, keep 1,268,211 and 3,850,916 paths, against 158,230 for Mumford1 (Figure 3). $P = 80$ keeps 228,940 and 315,170 paths, and $P = 40$ keeps 132,883 and 177,841, with menu terms 0.4791 and 0.6942. No $P = 80$ root bound on Mumford2 exceeds 23.5349: the $P = 80$ restricted-master LP over a 22,961-route warm pool has this value, and adding columns can only lower it (interior-point solve without crossover). Our $P = 80$ bound 23.5115 lies below it; the gap at $P = 80$ stays at least 1.93%.

Table 12: Menu term (true ATT minus direct-menu value) at our best network, in minutes, with $K = 5$. Without a P cap, the 5,000-path safety cap (Section 4.3) interrupts 11 pairs on Mumford2, 154 on Mumford3 and none on Mumford0–1.

Instance	$P = 20$	$P = 40$	$P = 80$	No P cap
Mumford0	0.0656	0.0186	0.0041	0.0021
Mumford1	0.4656	0.2710	0.1459	0.0409
Mumford2	0.6752	0.4791	0.3057	0.0229
Mumford3	0.9001	0.6942	0.5076	0.0226

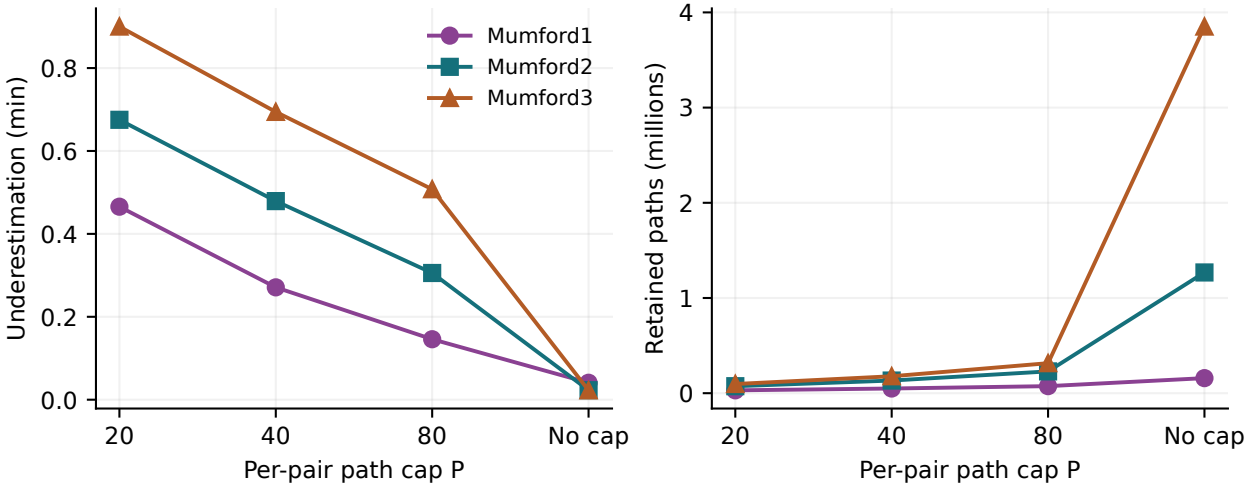


Figure 3: Effect of the per-pair cap on the menu term and on retained paths, Mumford1–3 (Mumford0, whose values are below 0.07 minutes, is in Table 12).

Table 13: Completed Mumford0 lower-bound trees: Δ (TTT) above the root bound 13.7202, certified LB in ATT. Both trees ran on 8 workers and branch on routes, groups and classes. Worker-s: summed node wall times (no CPU time recorded); margin: smallest certified leaf bound minus cutoff (TTT); verify: re-verification wall seconds.

Δ	LB	Nodes	Wall (s)	Worker-s	Margin	Verify (s)
12,000	13.7553	2,509	10,764	85,509	0.0208	49
15,000	13.7641	14,471	88,868	708,519	0.0011	1,358

One-transfer options (SL) leave these menu terms unchanged. On Mumford0 all three $K = 5$ menus underestimate the incumbent by only 730 TTT (0.0021 minutes). The PL menu with $K = 10$, or one extra length group for the pairs $(14, 21)$ and $(15, 27)$, removes this.

8.5. Completed searches on Mumford0

On Mumford0 the menu term is only 2.6% of the root gap (Table 11), so search is the lever.

The bound trees start from the PL $K = 5$ root certificate and run without an incumbent at cutoff $\Lambda_{\text{root}} + \Delta$ (Section 7.4); every pruned leaf carries a threshold-enumeration certificate. The tree at $\Delta = 15{,}000$ TTT (Table 13) proves $\text{TTT}(R) > 4{,}709{,}532$ for every feasible network. This raises the bound from 13.7202 to 13.7641 and cuts the gap to 0.28%. Smaller trees ($\Delta \leq 9{,}000$, older drivers) are dominated and not used; their records are in the data archive.

Table 14: Mumford0 exchange neighbourhoods: for each k, the number of k-subsets of the incumbent's routes, the share contained in some certified released set, and the number of released sets of size k.

k	k-subsets	Coverage (%)	Released sets
1	12	100.0	0
2	66	100.0	0
3	220	100.0	0
4	495	100.0	0
5	792	100.0	0
6	924	82.4	12
7	792	50.3	0
8	495	23.8	30
9	220	9.5	2
10	66	3.0	2

Table 15: Mumford0 expansion of Gao et al. (2026, Table 10): Gao et al.'s best TTT and gap (LBG), and our certified optimum (closed trees; both pass the verifier).

S	Gao TTT	Gao LBG (%)	Optimum TTT	ATT	Nodes	Wall (s)
12+1	4,743,820	0.00	4,743,820	13.8643	3	77
12+2	4,707,830	0.62	4,707,830	13.7591	13	155

Forty-six certified released sets cover every subset of at most five of the incumbent's twelve routes (Table 14). By Corollary 1, every strictly better network replaces at least six routes, beyond the one- and two-route kicks of our local search. Replacements range over all legal routes. The 46 trees have 380 nodes (43 pruned, 13 infeasible and 157 finisher-closed leaves); solving took 19,443 wall seconds and verification 2,287. They evaluated 47 networks, excluded 1 and raised the depth map 46 times (92 pairs). Their smallest margins (0.019 TTT at pruned and 0.041 TTT at finisher leaves) are positive, as are those of the trees of Table 13, so all of them satisfy the strict acceptance rule of Section 7.5. The trees of Table 13 met no integral node LP.

The same trees solve the expansion problem of Gao et al. (2026, Table 10), which forces the twelve routes of the published network net0 of Wens and Vansteenwegen (2025) and adds one or two routes (Table 15). With one added route, our optimum 4,743,820 confirms their value and zero gap. With two, our optimum 4,707,830 equals their value, so their network is optimal and their 0.62% gap is closed. Each claim covers every network of 13 or 14 distinct legal routes containing the twelve base routes.

8.6. Status of the Mumford0 closure run (as of 22 October 2026)

A global tree with all twelve routes free and cutoff 4,722,420 TTT, ten below the incumbent, has been running for 15 days on 56–64 cores. If it completes and passes verification, it closes Mumford0 (Theorem 1(a)). It had solved 972,356 nodes, with 20,029 open, and created 9.9×10^5 nodes; it solves about 6.7×10^4 per day. The created nodes are 20–28% of the 3.5×10^6–5.0×10^6 nodes estimated by stratified tree-size sampling (Chen, 1992) with 3 seeds. At this rate it would complete between about 29 November and 21 December 2026; this is a computed projection, not a bound. The smallest bound over its open nodes, 13.7624, is still below the bound of Section 8.5 (computed). A revised version will report the outcome.

9. Conclusions

How far are the best Mumford networks from optimal? Each Mumford instance now has an interval for its optimal passenger cost, from a certified bound to a computed network value, with gaps from 0.28% to 2.79%. On Mumford0, every strict improvement must change at least half of the incumbent's routes. Short path lists, automaton pricing with a computable pricing bound, complete pools, and exact evaluation with refinement and exclusion made this possible without listing the route universe.

On Mumford2 and Mumford3 the truncated menus now limit the bounds: at our networks they account for about two thirds of the gap. Adaptive per-pair menus guided by the menu term, and stronger transfer underestimators, are the natural next step. Verification uses floating-point arithmetic with explicit tolerances, unlike exact verification of integer programs (Cheung et al., 2017); rational or interval reconstruction (Neumaier and Shcherbina, 2004) would strengthen it. The benchmark omits capacities and frequency-dependent waiting. Extending the method to them needs new passenger underestimators and pricing coefficients.

Appendix A. Accounting for route upper bounds and fixed routes

The restricted master's upper-bound multipliers belong to its explicit columns and do not apply to the implicit routes. Pricing may therefore omit a finite set J of free columns, for example columns at their upper bounds, and score them separately. Let F be the forced routes, which contribute $\sum_{r\in F}\bar{c}_r$ as in Proposition 2, and suppose every free route not in J has reduced cost at least δ. For a feasible selection, with $\sum_{r\text{ free}} z_r = S - |F|$,

$$\sum_{r\text{ free}} \bar{c}_r z_r \geq \sum_{r\in J} \bar{c}_r z_r + \delta\Big(S - |F| - \sum_{r\in J} z_r\Big) \geq (S-|F|)\delta + \sum_{r\in J} \min\{0, \bar{c}_r - \delta\}, \tag{A.1}$$

where the last step uses $0 \leq z_r \leq 1$ on J. A certificate lists the omitted routes. Pricing every route and applying Proposition 2 gives a simpler, possibly weaker bound.

Appendix B. Verification and reproducibility

Each stored network and certificate carries what a separate check needs to re-derive it. A feasible route set is checked for route count, distinctness, simple paths, length bounds, all-node coverage and connectivity before its costs are accepted. A root certificate specifies the instance, path limits, dual vector, pricing evidence and menu variant, since rebuilding the wrong variant changes the meaning of the dual. A tree certificate records each decision (route, group or class, with the class members' hash), each leaf's cutoff, depth map and certificate, the forced routes of a neighbourhood root and every evaluated network. Beyond the checks of Section 7.8, the verifier checks that sibling decisions are complementary on the same object, re-enumerates class members, re-checks every leaf at its own cutoff and map, re-evaluates every listed network and checks $\tau_{\text{leaf}} \geq \tau_f$. A split tree is accepted only when the parts' frontiers are disjoint and their union is the parent's open frontier. An incomplete enumeration, a missing leaf or a parked node cannot establish closure. Negative controls that inflate a root bound, a box bound or a finisher cut are rejected.

The neighbourhood trees, the expansion trees and the closure run share these settings: initial box radius 10 with growth factor 4, enumeration cap 5×10^6 routes (22 neighbourhood trees: 1.2×10^7) and 900 seconds, node time 1,200 seconds, 60 box steps with 60 inner iterations, class thresholds at fractions 0.9, 0.8, 0.7, 0.6 and 0.5 of $|G_0|$, parent-side certificates with margin 0.02 and two repair rounds, finisher gap 4,000, pool cap 1.5×10^6, and a finisher limit of 7,200 seconds or 20,000

nodes. The closure run adds hybrid node order with 20,000 open nodes, finisher cuts, and the stall and hold rules below; the Mandl trees of Section 8.2 use its driver and rules (`de73638` with the fixes `f81a85d` and `b4bae9e`). Its code changed while it ran: centre handed to children (12 October 2026), stall rule and finisher cuts (13 Oct.), hold rule and re-queue of stalled nodes (14 Oct.), forced branching at caps (15 Oct.), exclusion of class candidates decided on the path (21 Oct., `1ebebaa`; later `f558ae6`, no other tree-code change). The two trees of Table 13 came from an earlier driver with the same box settings, class fractions 0.9, 0.8 and 0.7, and no parent-side certificates, preview centres or finisher. Command lines and arguments are archived with each tree.

The limits of Section 7.6 work as follows. A node stops its box steps at an enumeration cap, a node time limit or a step limit. Caps count routes in both orientations and are checked periodically, so they are approximate. On the closure run a stall rule also stops them when the last three steps together raised the bound by less than 0.5% of the remaining gap, unless the gap is just above the finisher threshold. The node keeps its best certified bound. A stalled node without a branching candidate is re-queued once with the rule off. A node stopped by a cap without a scored candidate is branched on the free route with the largest fractional, else largest positive, LP value. A node that did not stall and returns neither a certificate nor a branching (a worker error, or a cap before the first LP) is parked at once. Otherwise a node is parked after more than 60 re-queues, more than two task time-outs (six hours) or more than two worker kills or deaths. Parked nodes are retried at a manual restart. After a failed finisher hand-off at gap Γ_0 (time or node limit, pool cap, error), the closure run hands descendants to the finisher only once their gap is at most $\Gamma_0 - 200$ (hold rule).

A separate check rebuilt all 1,126 finisher cuts of a test tree in exact integer arithmetic and confirmed each by a mixed-integer program over the node's integral points (for pools above 5,000 routes, over a relaxation containing them). For 11 cuts it relies on the solver's dual bound at its 600-second limit. The check ran single-threaded; the LP solver interface's thread scheduler failed only in multi-threaded sessions. Its records are in the data archive.

The data archive holds instance summaries, published-route checks, our route sequences, menu experiments, root-certificate check records, tree and neighbourhood evidence and experiment provenance. Every table and macro of this paper is generated from these files. Source hashes identify the records used, and the build stops if one has changed. The ordinary and stabilized phases of the cold root campaigns and the Mumford1 box phase ran at commits `9eb4a00` and `0eef091`, and the trees of Table 13 at `d788036` and `27ccdc7`. The cold Mumford2–3 box phases, the first root-certificate checks and 89 of the first 100 rows of the experiment ledger (fixed-network, formulation and menu experiments) ran from a working tree with uncommitted changes relative to commit `1be9329`. These certificates were re-verified at a committed version. The Mumford2 ordinary $P = 80$ phase ran at `a9df517`, all other $P = 80$ phases at `de73638`. The ledger keeps stopped and unsuccessful runs.

The data snapshot was finalized on 22 October 2026. Its re-verification (root, tree and released-set certificates, exchange covering) passed 54 of 54 checks at `d57f7be` (3,156 s), the 6 $P = 80$ certificates and 2 chains passed at `47936a4`, and the expansion trees when produced (`4c4ab5a`). Exact recomputation of 51 pruned leaves (8 of Table 13, all of the released sets) and all 1,186 finisher leaves moves no bound by more than 7.9×10^{-10} TTT; every exact margin is at least 0.0011 TTT.

Appendix C. Pricing and stabilization settings

In the root campaigns the initial suffix memory contains the last five stops, and detected repeated-node segments are inserted as memory strings. At an optimizing walk ϖ, the subgradients of U in (15) are $\psi_v = 1 - n_v(\varpi)$ and $\psi_g = 1 - n_g(\varpi)$. After suppressing components that

would move a zero multiplier below zero, the projected update is

$$(\nu, \xi) \leftarrow [(\nu, \xi) - \eta\psi]_+, \qquad \eta = \phi \, \frac{\max\{U(\nu, \xi) - \Pi^{\mathrm{tgt}}, 1\}}{\|\psi\|_2^2},$$

where Π^{tgt} is the largest of zero, the best elementary prize and any requested pricing threshold. The step factor starts at $\phi = 0.5$ and is halved after five steps without improvement, with a restart from the best recorded multipliers. Multipliers are warm-started between calls. Validity follows from nonnegativity and the recomputed dynamic program, whatever the step rule.

Stabilized column generation prices at $y^s = \omega y^c + (1 - \omega) y^{LP}$, with y^c the best certified centre. After the k-th consecutive mis-pricing (no column added), $\omega_k = \max\{0, 1 - k(1 - \omega_0)\}$ with $\omega_0 = 0.8$, until the restricted-master dual itself is used. A successful pricing resets ω to ω_0. In the cold pipeline, ordinary column generation runs for 3,600 seconds or 500 iterations, whichever comes first (Mumford2 stopped at the iteration limit). A stabilized phase follows, with a 57,600-second budget checked between iterations and 300 seconds and 400 subgradient iterations per pricing call. Box steps come last, starting at radius 5 with growth factor 4 and adapting the radii of binding rows. Table C.16 summarizes the pricing work of the campaigns.

Table C.16: Pricing statistics of the cold campaigns: iterations and exact certification calls of ordinary column generation, stabilized iterations, share of the stabilized phase spent in recorded exact pricing, memory strings in the stored pricer state, box steps and largest box pool (routes).

Instance	CG iter.	CG exact	Stab. iter.	Stab. pricing (%)	Strings	Box steps	Max pool
Mumford1	163	87	387	78.5	532,561	2	1,861
Mumford2	500	174	5,810	7.1	29,379	13	13,767
Mumford3	413	178	5,713	10.7	50,566	18	1,340,747

Appendix D. Primal heuristic and provenance

This appendix records how our networks were found; all comparisons use the stored route sets, re-evaluated in Python.

Feasible networks come from iterated local search over the full simple-path space. Local proposals trim and regrow a route end, reconnect a short internal segment, or exchange the tails of two routes at a common stop. A direct-ride lower bound screens proposals before exact evaluation. Each outer iteration removes one or two routes, keeps a random segment of 3–8 stops of each and regrows it, randomly or by a surrogate-guided beam search. It then runs a Metropolis quench (temperature from 200 down to 5 TTT) and accepts the quenched network by a Metropolis test at temperature 300 TTT. The quench uses 10^6 proposals on Mumford1 and 3×10^5 on Mumford2 and Mumford3.

The Mumford1–3 searches started from the best public networks of Wens and Vansteenwegen (2025). They ran ruin-and-recreate neighbourhood search, a short calibration and three final chains of 7,200 seconds with distinct seeds (base seeds 5101, 5201, 5301; chain c adds $c \cdot 1{,}000{,}000{,}007$), all from one shared incumbent (Table D.17). For Mumford0, coverage MIPs over a column-generation pool and neighbourhood search gave 13.8360. A separate campaign (ruin-and-recreate evolution over route pools, 12-route MIPs and exact three-route descent) produced 4,723,960. Iterated local search with one- and two-route kicks improved it to 4,723,820 and then 4,722,430. The campaign's primal ledgers record 6.5 and 41.0 wall hours for its two stages. Twelve cold starts of at least 45 minutes did not reach 4,722,430; the best reached 4,723,490 (Table D.18).

Table D.17: Final improvement chains (three per instance, 7,200 s each, shared start).

Instance	Best ATT	Mean ATT	Worst ATT	SD
Mumford1	21.205709	21.206278	21.207417	0.000986
Mumford2	23.999705	24.000359	24.001163	0.000741
Mumford3	26.737154	26.738273	26.739448	0.001148

Table D.18: Logged primal runs of our main campaign (wall hours) and start networks. The separate Mumford0 campaign is not included.

Instance	Logged runs	Wall (h)	Start
Mumford0	91	8.0	CG-pool coverage MIPs
Mumford1	8	7.5	Wens & Vansteenwegen
Mumford2	6	6.4	Wens & Vansteenwegen
Mumford3	5	6.4	Wens & Vansteenwegen

Acknowledgements

The author thanks Zhaojiacheng Zhou for providing a computing server.

Declaration of competing interest

The author declares no competing interests.

Funding

This research received no specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data and code availability

Source code, benchmark instance files, feasible route sets, experiment records, certificates and the verification tools are archived on Zenodo, doi:10.5281/zenodo.23523785.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the author used OpenAI Codex and Anthropic Claude Code in order to assist with coding and analysing experiments, and revising the manuscript. After using these tools, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article.

References

Ahmed, L., Mumford, C., Kheiri, A., 2019. Solving urban transit route design problem using selection hyper-heuristics. European Journal of Operational Research 274, 545–559. doi:10.1016/j.ejor.2018.10.022.

Aho, A.V., Corasick, M.J., 1975. Efficient string matching: An aid to bibliographic search. Communications of the ACM 18, 333–340. doi:10.1145/360825.360855.

Aktaş, D., Vermeir, E., Vansteenwegen, P., 2024. Speeding up the passenger assignment problem in transit planning by the direct link network representation. Computers & Operations Research 167, 106647. doi:10.1016/j.cor.2024.106647.

Balas, E., Jeroslow, R., 1972. Canonical cuts on the unit hypercube. SIAM Journal on Applied Mathematics 23, 61–69. doi:10.1137/0123007.

Baldacci, R., Christofides, N., Mingozzi, A., 2008. An exact algorithm for the vehicle routing problem based on the set partitioning formulation with additional cuts. Mathematical Programming 115, 351–385. doi:10.1007/s10107-007-0178-5.

Baldacci, R., Mingozzi, A., Roberti, R., 2011. New route relaxation and pricing strategies for the vehicle routing problem. Operations Research 59, 1269–1283. doi:10.1287/opre.1110.0975.

Barnhart, C., Johnson, E.L., Nemhauser, G.L., Savelsbergh, M.W.P., Vance, P.H., 1998. Branch-and-price: Column generation for solving huge integer programs. Operations Research 46, 316–329. doi:10.1287/opre.46.3.316.

Borndörfer, R., Grötschel, M., Pfetsch, M.E., 2007. A column-generation approach to line planning in public transport. Transportation Science 41, 123–132. doi:10.1287/trsc.1060.0161.

Borndörfer, R., Karbstein, M., 2012. A direct connection approach to integrated line planning and passenger routing, in: Delling, D., Liberti, L. (Eds.), 12th Workshop on Algorithmic Approaches for Transportation Modelling, Optimization, and Systems (ATMOS 2012), OpenAccess Series in Informatics (OASIcs), vol. 25, pp. 47–57. doi:10.4230/OASIcs.ATMOS.2012.47.

Cancela, H., Mauttone, A., Urquhart, M.E., 2015. Mathematical programming formulations for transit network design. Transportation Research Part B: Methodological 77, 17–37. doi:10.1016/j.trb.2015.03.006.

Caprara, A., Fischetti, M., 1996. {0, 1/2}-Chvátal–Gomory cuts. Mathematical Programming 74, 221–235. doi:10.1007/BF02592196.

Ceder, A., Wilson, N.H., 1986. Bus network design. Transportation Research Part B: Methodological 20, 331–344. doi:10.1016/0191-2615(86)90047-0.

Chen, P.C., 1992. Heuristic sampling: A method for predicting the performance of tree searching programs. SIAM Journal on Computing 21, 295–315. doi:10.1137/0221022.

Cheung, K.K.H., Gleixner, A., Steffy, D.E., 2017. Verifying integer programming results, in: Eisenbrand, F., Koenemann, J. (Eds.), Integer Programming and Combinatorial Optimization (IPCO 2017), Lecture Notes in Computer Science, pp. 148–160. doi:10.1007/978-3-319-59250-3_13.

Christofides, N., Mingozzi, A., Toth, P., 1981. State-space relaxation procedures for the computation of bounds to routing problems. Networks 11, 145–164. doi:10.1002/net.3230110207.

Chu, J.C., 2018. Mixed-integer programming model and branch-and-price-and-cut algorithm for urban bus network design and timetabling. Transportation Research Part B: Methodological 108, 188–216. doi:10.1016/j.trb.2017.12.013.

Duan, N., Günlük, O., Samaranayake, S., 2026. Integrated multi-modal transit network design via dynamic line generation. URL: https://arxiv.org/abs/2608.26444, arXiv:2608.26444. Preprint.

Durán-Micco, J., Vansteenwegen, P., 2022. A survey on the transit network design and frequency setting problem. Public Transport 14, 155–190. doi:10.1007/s12469-021-00284-y.

Fan, L., Mumford, C.L., 2010. A metaheuristic approach to the urban transit routing problem. Journal of Heuristics 16, 353–372. doi:10.1007/s10732-008-9089-8.

Fischetti, M., Lodi, A., 2003. Local branching. Mathematical Programming 98, 23–47. doi:10.1007/s10107-003-0395-5.

Fiss, B.C., Ritt, M., 2012. Exact and metaheuristic algorithms for the urban transit routing problem, in: XLIV Simpósio Brasileiro de Pesquisa Operacional, pp. 3717–3728. URL: https://www.din.uem.br/sbpo/sbpo2012/pdf/arq0328.pdf.

Gao, W., Ma, J., He, Y., Dong, C., 2026. An exact approach for the transit route network design problem. European Journal of Operational Research 334, 111–127. doi:10.1016/j.ejor.2026.04.020.

Goerigk, M., Schmidt, M., 2017. Line planning with user-optimal route choice. European Journal of Operational Research 259, 424–436. doi:10.1016/j.ejor.2016.10.034.

Guihaire, V., Hao, J.K., 2008. Transit network design and scheduling: A global review. Transportation Research Part A: Policy and Practice 42, 1251–1273. doi:10.1016/j.tra.2008.03.011.

Hansen, S.M.C., Hoogervorst, R., Nielsen, O.A., Lusby, R.M., van der Hurk, E., 2026. An exact algorithm for public transport line planning considering passenger and operational costs and lost demand. European Journal of Operational Research doi:10.1016/j.ejor.2026.06.034. In press.

Heinrich, I., Schiewe, P., Seebach, C., 2022. Line planning in public transport: Bypassing line pool generation. URL: https://arxiv.org/abs/2205.09580, arXiv:2205.09580. Preprint.

Holliday, A., El-Geneidy, A., Dudek, G., 2025. Learning heuristics for transit network design and improvement with deep reinforcement learning. Transportmetrica B: Transport Dynamics 13, 2561863. doi:10.1080/21680566.2025.2561863.

Huangfu, Q., Hall, J.A.J., 2018. Parallelizing the dual revised simplex method. Mathematical Programming Computation 10, 119–142. doi:10.1007/s12532-017-0130-5.

Hüsselmann, G., van Vuuren, J., Andersen, S., 2024. An improved solution methodology for the urban transit routing problem. Computers & Operations Research 163, 106481. doi:10.1016/j.cor.2023.106481.

John, M.P., Mumford, C.L., Lewis, R., 2014. An improved multi-objective algorithm for the urban transit routing problem, in: Evolutionary Computation in Combinatorial Optimisation (EvoCOP 2014), Lecture Notes in Computer Science, vol. 8600, pp. 49–60. doi:10.1007/978-3-662-44320-0_5.

Kılıç, F., Gök, M., 2014. A demand based route generation algorithm for public transit network design. Computers & Operations Research 51, 21–29. doi:10.1016/j.cor.2014.05.001.

Knuth, D.E., 1975. Estimating the efficiency of backtrack programs. Mathematics of Computation 29, 121–136. doi:10.1090/S0025-5718-1975-0373371-6.

de Lima, V.L., Iori, M., Miyazawa, F.K., 2023. Exact solution of network flow models with strong relaxations. Mathematical Programming 197, 813–846. doi:10.1007/s10107-022-01785-9.

Lübbecke, M.E., Desrosiers, J., 2005. Selected topics in column generation. Operations Research 53, 1007–1023. doi:10.1287/opre.1050.0234.

Mandl, C.E., 1979. Applied network optimization. Academic Press, London.

Mandl, C.E., 1980. Evaluation and optimization of urban public transportation networks. European Journal of Operational Research 5, 396–404. doi:10.1016/0377-2217(80)90126-5.

Marsten, R.E., Hogan, W.W., Blankenship, J.W., 1975. The BOXSTEP method for large-scale optimization. Operations Research 23, 389–405. doi:10.1287/opre.23.3.389.

Martínez-Quezada, D.O., Cortés, C.E., Gendreau, M., Mauttone, A., 2026. A bilevel model and a column generation heuristic for the transit network design and frequency setting problem with capacity constraints. Transportation Research Part B: Methodological 211, 103542. doi:10.1016/j.trb.2026.103542.

du Merle, O., Villeneuve, D., Desrosiers, J., Hansen, P., 1999. Stabilized column generation. Discrete Mathematics 194, 229–237. doi:10.1016/s0012-365x(98)00213-1.

Mumford, C.L., 2013. New heuristic and evolutionary operators for the multi-objective urban transit routing problem, in: 2013 IEEE Congress on Evolutionary Computation, pp. 939–946. doi:10.1109/cec.2013.6557668.

Neumaier, A., Shcherbina, O., 2004. Safe bounds in linear and mixed-integer linear programming. Mathematical Programming 99, 283–296. doi:10.1007/s10107-003-0433-3.

Nikolić, M., Teodorović, D., 2013. Transit network design by bee colony optimization. Expert Systems with Applications 40, 5945–5955. doi:10.1016/j.eswa.2013.05.002.

Owais, M., 2026. Transit network design problem: A half century of methodological research. Innovative Infrastructure Solutions 11, 3. doi:10.1007/s41062-025-02356-5.

Pessoa, A., Sadykov, R., Uchoa, E., Vanderbeck, F., 2013. In-out separation and column generation stabilization by dual price smoothing, in: Experimental Algorithms (SEA 2013), Lecture Notes in Computer Science, vol. 7933, pp. 354–365. doi:10.1007/978-3-642-38527-8_31.

Pessoa, A., Sadykov, R., Uchoa, E., Vanderbeck, F., 2018. Automation and combination of linear-programming based stabilization techniques in column generation. INFORMS Journal on Computing 30, 339–360. doi:10.1287/ijoc.2017.0784.

Righini, G., Salani, M., 2008. New dynamic programming algorithms for the resource constrained elementary shortest path problem. Networks 51, 155–170. doi:10.1002/net.20212.

Schmidt, M., Schöbel, A., 2015. The complexity of integrating passenger routing decisions in public transportation models. Networks 65, 228–243. doi:10.1002/net.21600.

Vanderbeck, F., Wolsey, L.A., 1996. An exact algorithm for IP column generation. Operations Research Letters 19, 151–159. doi:10.1016/0167-6377(96)00033-8.

Vermeir, E., Engelen, W., Philips, J., Vansteenwegen, P., 2021. An exact solution approach for the bus line planning problem with integrated passenger routing. Journal of Advanced Transportation 2021, 6684795. doi:10.1155/2021/6684795.

Wens, M., Vansteenwegen, P., 2025. A decentralised genetic algorithm with sub-objectives for the transit network design problem. Computers & Operations Research 183, 107191. doi:10.1016/j.cor.2025.107191.

Wentges, P., 1997. Weighted Dantzig–Wolfe decomposition for linear mixed-integer programming. International Transactions in Operational Research 4, 151–162. doi:10.1111/j.1475-3995.1997.tb00071.x.

Yamagishi, P.M.F., Fampa, M., Lee, J., 2026. The dual-path fixing strategy and its application to the set-covering problem, in: 24th International Symposium on Experimental Algorithms (SEA 2026), Leibniz International Proceedings in Informatics (LIPIcs), vol. 371, pp. 28:1–28:18. doi:10.4230/LIPIcs.SEA.2026.28.

Yang, Y., 2025. DeLuxing: Deep Lagrangian underestimate fixing for column-generation-based exact methods. Operations Research 73, 1184–1207. doi:10.1287/opre.2023.0398.